METERMIND

Electricity Billing Detection and Consumer Assistant

Full 38-Day Project Specification — iamneo Capstone

PROBLEM STATEMENT

MeterMind is an electricity distribution company that bills twenty-five thousand low-tension connections across five divisions. Every billing cycle a meter reader records a reading, and the company prices it by hand against a tariff order that changes each financial year. Its billing clerks apply the slab table, the fixed charge, the electricity duty and the rebate in whatever order each of them learned, and a reading that cannot have happened is argued over rather than refused. Its vigilance team has no way to rank which of five thousand suspect readings deserves an inspection first, and a case wrongly sent to vigilance costs the company a consumer. Its grievance desk opens every complaint itself, and the wording that decides a bill is published across twelve separate tariff orders, several of which have been superseded while the versions they replaced still govern the cycles billed under them.

The company requires a single platform that prices a reading against the schedule in force, serves that domain over an authenticated web service, sorts suspect readings by what a trained model expects of them, reads incoming complaints without a person opening each one, answers a question out of the tariff order that was actually in force in the billing period asked about, and puts a consumer assistant behind one endpoint that can reach every earlier day as a tool and stop for an inspector when it should.

OBJECTIVE

Build the **MeterMind** platform across thirty-eight days, each day extending what the previous days produced.

Begin with the billing domain in plain Python — the units a meter recorded, the telescopic and non-telescopic slab tables, the fixed charge on the sanctioned load, the electricity duty and the means-tested rebate — and grow it into a package with validated entities and a connection hierarchy. Clean the reading book into a modelling dataset, load it into a database, and expose the domain through a FastAPI service protected by token authentication, role rules and a case status ladder.

Train and compare exception classification models on a shared preparation pipeline, choose a champion by cross-validation, derive three columns the book does not carry, choose an operating rule on cost rather than on accuracy, and sort the flagged readings into the five queues a distribution company works.

Turn the complaint corpus into structured fact: clean it, classify category and urgency with a trained model, score sentences by attention to summarise a thread, and send that summary to a language model for a reading no counting can produce. Extract the account numbers and meter serials a consumer quotes, check them against the register, run the corpus in batch, and escalate the complaints a supervisor must see today.

Index the tariff orders into a ChromaDB vector store with the metadata that dates them, answer questions only from retrieved extracts with every citation resolved, and narrow retrieval to the order in force in the billing period asked about. Finally, wrap four earlier days as tools, drive them with a graph of agents that triage, plan, gather, draft, check and answer, hold a run for an inspector where the platform must not answer alone, and expose the whole assistant as an authenticated endpoint on the service built in weeks two and three.

FILE STRUCTURE

The original spec shows the file structure as two screenshot images (hosted externally, not reproducible here). Reconstructed below from the actual implementation built against this spec.

```
Project/
├── config/
│   └── constants.py
├── scripts/
│   ├── day01_bill_from_reading.py
│   └── day02_bill_pricing.py
├── metermind/
│   ├── core/
│   │   ├── exceptions.py
│   │   ├── tariff_calculations.py
│   │   ├── entities.py
│   │   ├── decorators.py
│   │   └── validators.py
│   ├── data_access/
│   │   ├── file_storage.py
│   │   ├── cleaning_pipeline.py
│   │   ├── database_loader.py
│   │   └── database_reports.py
│   ├── api/
│   │   ├── schemas.py
│   │   ├── application_factory.py
│   │   ├── repository.py
│   │   ├── sql_repository.py
│   │   ├── orm_models.py
│   │   ├── middleware.py
│   │   ├── password_hashing.py
│   │   ├── access_control.py
│   │   ├── case_transitions.py
│   │   └── routers/
│   │       ├── readings.py
│   │       ├── user_accounts.py
│   │       ├── screening.py
│   │       └── assistant.py
│   ├── exception_models/
│   │   ├── feature_pipeline.py
│   │   ├── baseline_model.py
│   │   ├── tree_models.py
│   │   ├── cross_validation.py
│   │   ├── feature_engineering.py
│   │   ├── champion_model.py
│   │   ├── investigation_queues.py
│   │   └── exception_report.py
│   ├── message_intelligence/
│   │   ├── attention.py
│   │   ├── text_cleaning.py
│   │   ├── complaint_classifier.py
│   │   ├── thread_summary.py
│   │   ├── llm_client.py
│   │   ├── reference_extraction.py
│   │   ├── batch_pipeline.py
│   │   └── escalation.py
│   ├── retrieval/
│   │   ├── document_loader.py
│   │   ├── chunking.py
│   │   ├── vector_store.py
│   │   ├── grounded_answer.py
│   │   └── filtered_search.py
│   └── assistant/
│       ├── state.py
│       ├── nodes.py
│       ├── agents.py
│       ├── tools.py
│       ├── graph.py
│       └── workflow.py
├── data/                    (read-only, supplied)
├── processed_data/          (generated)
├── database/                (generated)
├── artifacts/               (generated)
├── output/                  (generated)
└── tests.py
```

DATASET DESCRIPTION

Every file below is supplied in the **data** folder, is read only, and is never written to by any day.

DATASET INVENTORY

File	Rows / Size	Used by
reading_book_train.csv	30,000 rows (28,021 survive cleaning)	Weeks 2–5
reading_book_predict.csv	5,000 rows	Week 4 Day 5
connection_register.csv	one row per connection	Week 6 Day 3
complaint_messages.csv	5,000 rows	Weeks 5–6
message_threads.csv	600 threads, 2,693 turns	Week 6 Day 1
tariff_orders (12 PDFs)	4 financial years × 3 categories, 2 amendments	Weeks 6–7
tariff_orders_manifest.json	12 entries	Weeks 6–7
retrieval_evaluation_set.json	148 questions	Week 7

reading_book_train.csv

The reading book with the exception attached, used to train and compare the models of Weeks 3 to 5. The file is deliberately unclean: it carries four categories the company does not bill at all — **temp**, **misc**, **streetlight** and **industrial**, read dates in four different shapes and sometimes not at all, identifiers that fail their published format, cycle lengths no cycle may run, and consumption no sanctioned load could draw. Cleaning it is the task of Week 2 Day 1, after which **28,021** rows survive and **1,979** are rejected with the reason each was rejected for.

Original SCHEMA and SAMPLE RECORDS given as screenshot images. Reconstructed below from the field names stated explicitly elsewhere in this spec (ReadingSubmission, ScreeningRequest, REQUIRED_FIELDS, and the modelling column lists).

Column	Type	Notes
account_number	text	shape ACC10000000
meter_serial	text	published serial format
consumer_category	text	domestic / commercial / agricultural, plus 4 unbilled categories present pre-clean
division, subdivision	text	one of five divisions
billing_period	text	YYYY-MM
read_date	text	4 different raw shapes
read_type	text	actual / estimated
sanctioned_load_kw	float	
cycle_days	integer	
previous_reading, current_reading	float	
units_consumed	float	
twelve_month_average_units	float	
same_month_last_year_units	float	
consecutive_estimated_reads	integer	
days_since_actual_read	integer	
neighbourhood_average_units	float	
arrears_inr	float	
payment_defaults_last_year	integer	
seal_check_result	text	
meter_make	text	
meter_install_date	text	
cycle_end_date	text	
exception_class	text	the label — one of 5 published classes; absent in reading_book_predict.csv

reading_book_predict.csv

The readings waiting to be classified, carrying every column of the training book except **exception_class**, which is the column the model exists to supply. Identifiers run from **RDG30000000**. Week 4 Day 5 sorts these into the five investigation queues.

connection_register.csv

Every connection on the books, one row each, keyed by account number. Week 6 Day 3 checks against it the account numbers and meter serials a consumer quotes in a complaint.

Column	Type	Notes
account_number	text	primary key
consumer_name	text	
consumer_category	text	
tariff_code	text	
sanctioned_load_kw	float	
meter_serial	text	unique across the register
division, subdivision	text	

complaint_messages.csv

One complaint each, as received. The text carries the quoted reply, the greeting, the sign-off and the signature block a real message carries, and removing them is the task of Week 5 Day 4. Both label columns are used by the classifiers of Week 5 Day 5.

Column	Type	Notes
message_id	text	e.g. MSG000002
account_number	text	
received_date	text	
message_text	text	raw, unwrapped
category	text	high_bill / no_supply / meter_problem / connection_request / payment_issue
urgency	text	routine / priority / emergency

message_threads.csv

600 complaint threads across 2,693 turns, one row per turn, ordered by turn number within each thread. Week 6 Day 1 summarises a whole thread from these turns.

Column	Type	Notes
thread_id	text	
turn_number	integer	ordering within the thread
turn_text	text	

tariff_orders (PDFs)

Twelve tariff orders as PDF documents, four financial years by three consumer categories, with two mid-year amendments among them. Their windows tile 2024-04-01 to 2027-03-31 without a gap and without an overlap, so exactly one order is in force for any category on any date. Each carries the same twelve sections, among them ENERGY CHARGE, FIXED CHARGE, ELECTRICITY DUTY, SUBSIDY AND REBATE and UNAUTHORISED USE OF ELECTRICITY.

tariff_orders_manifest.json

A list of twelve entries, one per order. Week 6 Day 5 loads only the documents this file names, and Week 7 Day 3 reads effective_from and effective_to to decide which order was in force in the period asked about.

Field	Type	Notes
order_id	text	e.g. FY2024-25-LT1
title	text	
consumer_category	text	
file_name	text	
effective_from	text	date
effective_to	text	date

retrieval_evaluation_set.json

148 questions, each with the order and the section that answers it. 37 distinct question-and-category pairs are asked across different billing periods, so the same question has different answers depending on the period, and every one of them is answered by at least two different orders. Week 7 Days 2 and 3 are measured against this file.

Field	Type	Notes

question_id	text	
question	text	
consumer_category	text	
billing_period	text	
answering_order	text	which order_id answers it
answering_section	text	which section heading

WEEK 1

WEEK 1 DAY 1 TASK

MODULE: scripts/day01_bill_from_reading.py

A single script with no functions and no classes, written entirely at the top level of the file. It reads one meter reading from the terminal, prices it against the domestic schedule, prints the figures it derived, and ends with a decision followed by every rule the reading breaks.

The pricing order introduced here — the units the meter recorded, the telescopic slab table, the fixed charge on the sanctioned load, the electricity duty, then the rebate — governs the whole platform. It returns as named functions on Week 1 Day 2, moves into `metermind/core/tariff_calculations.py` on Week 1 Day 3, and is what the service of Week 2 answers with.

CONSTANTS FROM config/constants.py: None are imported. The schedule this day needs is the one config/constants.py already holds, in SLAB_TABLE_INR_BY_CATEGORY, FIXED_CHARGE_INR_PER_KW_BY_CATEGORY, ELECTRICITY_DUTY_PERCENT_BY_CATEGORY, SUBSIDY_INR_PER_UNIT_BY_CATEGORY and SUBSIDY_QUALIFYING_UNITS_BY_CATEGORY, and the file is supplied pre-loaded, so the figures can be read from it. On this day they are written into the script by hand; from Week 1 Day 3 the platform imports them.

SCRIPT: scripts/day01_bill_from_reading.py

FUNCTION SIGNATURE: None. The day has no functions and no classes; every statement is written at the top level of the file.

PARAMETERS: None. The script takes its seven values from standard input, described under INPUT AND OUTPUT below.

PURPOSE: Prices one domestic meter reading typed at the terminal and reports the figures it derived, the amount payable, and every reason the reading would be refused.

REQUIREMENTS:

- Read seven values from standard input, one to a line, in the order given under INPUT AND OUTPUT: account_number, sanctioned_load_kw, cycle_days, previous_reading, current_reading, arrears_inr and late_payment_surcharge_percent. Settle account_number to upper case with surrounding whitespace removed.
- Hold the domestic schedule by hand: four consumption blocks with their own limits and rates, a fixed charge for each kilowatt sanctioned, an electricity duty as a percentage of the energy charge, a rebate for each unit used where consumption qualifies for it, and the consumption at or under which it qualifies.
- Work out the most a connection of this load could physically draw across the cycle, from the sanctioned load, the hours in a day, the days in the cycle and the allowance a meter is given over its sanctioned load. This figure decides both a rollover and a refusal, so it is derived once.
- Report units_consumed as the difference between the two readings where the meter went forward. Where the current reading is below the previous one, the meter has either rolled past its last digit and started again or it has gone backwards, and only the first of those is billable: a rollover is credited when the previous reading plus what the connection could draw would have reached the meter's ceiling, and nothing else is.
- Charge the consumption TELESCOPICALLY: every block is charged at its own rate, and only the units falling inside a block are charged at that block's rate. A cycle of four hundred and sixty-seven units is not four hundred and sixty-seven units at the top rate.
- Report fixed_charge_inr from the sanctioned load rather than from the units, so a connection that used nothing still carries it.
- Report electricity_duty_inr as the published percentage of the energy charge alone, not of the whole bill.
- The rebate is means-tested by consumption. A cycle at or under the qualifying figure earns the per-unit rebate on every unit it used; a cycle one unit above it earns nothing at all.
- Report current_demand_inr as the energy charge plus the fixed charge plus the duty, less the rebate, and never below zero.
- Report the surcharge as the published percentage of the arrears, and amount_payable_inr as the current demand plus the arrears plus that surcharge.
- Print one labelled line for each of the nine reported figures, in this order: account_number, units_consumed, energy_charge_inr, fixed_charge_inr, electricity_duty_inr, the rebate, current_demand_inr, the surcharge and amount_payable_inr. Units are rounded to one decimal place and every rupee figure to two before printing, and each is printed as the number itself — a figure that rounds to 280.0 prints as 280.0, not as 280.00.
- Those nine are the figures the day is judged on. Whatever is worked out along the way to them — the units falling in each block, the physical maximum, the count of broken rules — is named as the writer of the script sees fit, and nothing later in the platform reads any of it.
- Refuse the reading when the meter went backwards without a rollover explaining it, when the consumption is above what the sanctioned load could physically draw, or when the billing cycle is outside the length a cycle may run.
- Print a separate line beginning Rejected: for every rule broken, each naming the figure it failed against and the limit it failed, so that the line can be read on its own without the reading beside it.
- Print exactly one closing line: Reading is billable where nothing is broken, and a line stating the reading is not billable together with how many reasons were found where something is.

RETURNS: Nothing. The result of the day is what the script prints.

> **INPUT:** Seven values typed at the terminal, one to a line, in this order. No data file is read.
> account_number – Text. The connection billed, in the shape ACC10000000.
> sanctioned_load_kw – Float. The load the connection is sanctioned for, in kilowatts.
> cycle_days – Integer. Days in the billing cycle.
> previous_reading – Float. What the meter showed at the start of the cycle.

current_reading – Float. What the meter shows now.
arrears_inr – Float. What the account already owes, in rupees.
late_payment_surcharge_percent – Float. The surcharge charged on those arrears, as a percentage.

CONSOLE INPUT:

```
ACC10000123
3
30
14208
14675
340
2
```

A domestic connection of 3 kW read over a 30 day cycle, the meter moving from 14208.0 to 14675.0, against arrears of 340.00 rupees carrying a 2 percent surcharge.

RUN COMMAND:

```
python3 scripts/day01_bill_from_reading.py
```

CONSOLE OUTPUT:

```
Account number: ACC10000123
Units consumed: 467.0
Energy charge: 2575.7
Fixed charge: 150.0
Electricity duty: 128.78
Rebate: 0.0
Current demand: 2854.48
Late payment surcharge: 6.8
Amount payable: 3201.28
Reading is billable
```

The transcript above is the format to produce. The test cases read the figures rather than the lines, so a label worded differently or spaced differently is not a failure — but every figure shown must appear, rounded as stated above and printed as the number itself. One line is read for its wording as well: a refusal of a reading that went backwards has to say so in words a reader can act on.

GENERATED FILES: None.

WEEK 1 DAY 2 TASK

MODULE: scripts/day02_bill_pricing.py

The same pricing as Week 1 Day 1, now expressed as six named calculations and one class rather than as a run of top-level statements, and covering all three consumer categories rather than the domestic one alone.

The class holds the reading and prices it. Its method derives its answer from the six calculations rather than repeating the arithmetic, so a change to a calculation reaches the class without being copied into it. All six calculations move into the package on Week 1 Day 3 and the class is replaced by the connection hierarchy on Week 1 Day 4; the names given here are the names those later days keep.

CONSTANTS FROM config/constants.py: None. This day declares its own constants at the top of the script: UNLIMITED_UNITS (integer, upper limit for a block with no limit), SLAB_RATES_INR (dict, block table per category — its keys are also the catalogue of billed categories), TELESCOPIC_CATEGORIES (tuple, which categories are charged block by block), FIXED_CHARGE_PER_KW_INR (dict), FIXED_CHARGE_PER_HORSEPOWER_INR (float), HORSEPOWER_PER_KILOWATT (float), ELECTRICITY_DUTY_PERCENT (dict), REBATE_PER_UNIT_INR (dict), REBATE_QUALIFYING_UNITS (dict), METER_CEILING_UNITS (integer). The same nine figures appear in config/constants.py under the names SLAB_TABLE_INR_BY_CATEGORY, FIXED_CHARGE_INR_PER_KW_BY_CATEGORY, FIXED_CHARGE_INR_PER_HORSEPOWER, HORSEPOWER_PER_KILOWATT, ELECTRICITY_DUTY_PERCENT_BY_CATEGORY, SUBSIDY_INR_PER_UNIT_BY_CATEGORY, SUBSIDY_QUALIFYING_UNITS_BY_CATEGORY, METER_ROLLOVER_UNITS and UNLIMITED_UNITS. The platform starts reading them on Week 1 Day 3.

FUNCTION: units_consumed(previous_reading, current_reading, physical_maximum_units)

PURPOSE: Returns the units the cycle used, reading a meter that has rolled past its last digit as a rollover rather than as a mistake.

REQUIREMENTS:

- Where the meter went forward, the units are the difference between the two readings.
- Where the current reading is below the previous one, two different things may have happened and only one of them is billable. Credit a rollover when the previous reading plus what the connection could physically draw would have reached the meter's ceiling, and count the units the meter recorded on both sides of that ceiling.
- Where no rollover explains it, the meter has gone backwards and the cycle used nothing.

RETURNS: A float, the units consumed.

FUNCTION: slab_energy_charge(consumer_category, units)

PURPOSE: Returns the energy charge for a cycle, under whichever of the two block conventions the category is billed by.

REQUIREMENTS:

- A TELESCOPIC category is charged block by block: only the units falling inside a block are charged at that block's rate, and every block below is charged at its own.
- A NON-TELESCOPIC category is charged at one rate: the whole consumption is charged at the rate of the block it lands in. One unit past a block boundary therefore reprices the entire cycle, and that is intended.
- Which convention a category is billed by is decided by TELESCOPIC_CATEGORIES, never by the category's name written into the arithmetic.
- A category with one block and one rate must come out the same way under either reading.

RETURNS: A float, the energy charge in rupees.

FUNCTION: apply_fixed_charge(consumer_category, sanctioned_load_kw)

PURPOSE: Returns the fixed charge the connection carries, from its sanctioned load rather than from what it used.

REQUIREMENTS:

- Two categories are billed for each kilowatt sanctioned, at their own published rate.
- The agricultural category is billed for each horsepower instead, so the sanctioned load is converted before the rate is applied. Billing an agricultural connection by the kilowatt is a different bill, not a rounding difference.
- A connection that used nothing at all still carries this charge.

RETURNS: A float, the fixed charge in rupees.

FUNCTION: apply_electricity_duty(consumer_category, energy_charge_inr)

PURPOSE: Returns the electricity duty the cycle carries.

REQUIREMENTS:

- The duty is the category's published percentage of the energy charge alone. It is not charged on the fixed charge and it is not charged on the whole bill.
- A category whose published duty is zero carries none.

RETURNS: A float, the duty in rupees.

FUNCTION: apply_subsidy(consumer_category, units)

PURPOSE: Returns the rebate the cycle earns, which is means-tested by consumption rather than tapered.

REQUIREMENTS:

- A cycle at or under the category's qualifying consumption earns the per-unit rebate on every unit it used.
- A cycle one unit above that figure earns nothing at all. There is no taper and no partial rebate.
- A category whose qualifying consumption is zero never earns one.

RETURNS: A float, the rebate in rupees.

FUNCTION: net_bill_amount(consumer_category, units, sanctioned_load_kw)

PURPOSE: Returns what the cycle costs, assembled from the four calculations above.

REQUIREMENTS:

- The four charges are obtained by calling the four calculations above, never by repeating their arithmetic here.
- The energy charge, the fixed charge and the duty are added, and the rebate is taken off.
- A bill is never negative. The agricultural category earns a rebate larger than its tariff, so a small agricultural cycle reaches zero and stops there.

RETURNS: A float, the amount in rupees.

CLASS: MeterReading

One reading, held as an object, able to price itself. Two members in all: a constructor and one method.

CONSTRUCTOR __init__(self, account_number, consumer_category, sanctioned_load_kw, cycle_days, previous_reading, current_reading): Holds one reading's fields as attributes of the object, under the same names the reading book uses. The six values are held as attributes under the names they were given. The same names are used by the file storage of Week 1 Day 3 and by the entities of Week 1 Day 4.

METHOD price(self): Returns what this reading used and what it costs. The most this connection could physically draw across the cycle is worked out from its own sanctioned load, the hours in a day, its own cycle length and the allowance a meter is given over its sanctioned load. The units and the amount are obtained by calling units_consumed and net_bill_amount, not by repeating what they do. Returns a tuple of two floats: the units consumed, and the amount in rupees.

MAIN BLOCK: Reads eight values from standard input, prices the reading through the class, and prints seven labelled lines. The block runs only when the file is run directly, so importing the module for its calculations prints nothing.

> **INPUT:** Eight values typed at the terminal, one to a line: account_number, consumer_category (settled to lower case), sanctioned_load_kw, cycle_days, previous_reading, current_reading, arrears_inr, late_payment_surcharge_percent.

CONSOLE INPUT:

```
ACC10000123
domestic
3
30
14208
14675
0
0
```

RUN COMMAND:

```
python3 scripts/day02_bill_pricing.py
```

CONSOLE OUTPUT:

```
Account number: ACC10000123
Consumer category: domestic
Units consumed: 467.0
Energy charge: 2575.7
Fixed charge: 150.0
Current demand: 2854.48
Amount payable: 2854.48
```

Run it a second time with a commercial connection of 30 kW whose meter moved from 99500 to 300 over a 30 day cycle, and the rollover is read as a rollover: the cycle used 800 units, not a negative number.

GENERATED FILES: None.

WEEK 1 DAY 3 TASK

The platform becomes a package. The pricing written twice as loose statements is moved into it once, under names every later week imports, and the first file is written to disk.

MODULE: metermind/core/exceptions.py

The error hierarchy the whole platform raises from. Every failure the platform is allowed to have is one of these, so a caller may catch the parent and be certain nothing else escapes.

CLASS: MeterMindError(Exception)

PURPOSE: The parent every error in the platform inherits from. Inherits from the built-in exception type and adds nothing to it. Catching this one catches every error the platform raises deliberately. Nothing in the platform raises a bare built-in exception where one of these would do.

CLASS: UnsupportedCategoryError(MeterMindError)

Raised where a consumer category has no published schedule. Inherits from MeterMindError and adds nothing to it.

CLASS: ReadingStorageError(MeterMindError)

Raised where a reading cannot be written or cannot be found. Both stores of Week 3 Day 1 and the service routes of Week 2 Day 4 raise it.

CLASS: InvalidReadingError(MeterMindError)

Raised where a reading cannot be true.

MODULE: metermind/core/tariff_calculations.py

The pricing of Week 1 Day 2, moved into the package and reading its figures from config/constants.py rather than declaring them. Three functions today; the proration pair arrives on Week 1 Day 5 and the whole module is what Week 2, Week 5 Day 1 and the assistant tool of Week 7 Day 4 price with.

PUBLISHED NAME: TELESCOPIC_CATEGORIES – Tuple. The categories charged block by block, declared at the top of this module. A category not named here is charged the other way.

FUNCTION: calculate_units_consumed(previous_reading, current_reading, physical_maximum_units)

Behaves as units_consumed of Week 1 Day 2 behaved, with the meter ceiling read from METER_ROLLOVER_UNITS rather than declared in the file. Returns a float, the units consumed.

FUNCTION: calculate_slab_energy_charge(consumer_category, units, blocks=None)

PARAMETERS: blocks – Tuple of pairs, optional. A block table to charge against instead of the published one. Where it is not supplied, the category's published table is used. Week 1 Day 5 supplies it with a prorated table.

REQUIREMENTS:

- A category with no published schedule raises UnsupportedCategoryError naming the category. It does not return zero and it does not fall back to another category's table.
- A TELESCOPIC category is charged block by block; a NON-TELESCOPIC category is charged at the single rate of the block its whole consumption lands in. Which convention applies is decided by TELESCOPIC_CATEGORIES.
- Where a block table is supplied, it is charged against in place of the published one, under the same convention the category is billed by.

RETURNS: A float, the energy charge in rupees.

FUNCTION: calculate_bill_amount(consumer_category, units, sanctioned_load_kw)

REQUIREMENTS:

- The energy charge comes from calculate_slab_energy_charge, never from arithmetic repeated here.
- The fixed charge follows the sanctioned load, by horsepower for the agricultural category and by kilowatt for the others.
- The duty is the category's published percentage of the energy charge alone.
- The rebate is means-tested by consumption exactly as on Week 1 Day 2.
- A bill is never negative.
- This is the one function the rest of the platform prices with. Week 2 Day 4, Week 5 Day 1 and the calculate_bill tool of Week 7 Day 4 all answer with what it returns.

RETURNS: A float, the amount in rupees.

MODULE: metermind/data_access/file_storage.py

The reading book on disk. One class today, gaining a third method on Week 1 Day 5.

CLASS: ReadingFileStorage

Reads and writes the reading book as CSV under one directory.

CONSTRUCTOR __init__(self, directory): Holds the folder every later call is resolved against, as a path object. The folder is not created here.

METHOD save_readings_csv(self, file_name, readings): Writes readings to a CSV file with a header row, and reports where it wrote them. An empty list raises ReadingStorageError. The folder is created if it is not there. The header is the column names of the first reading. Returns the path written.

METHOD load_readings_csv(self, file_name): Reads a reading book back from CSV. A file that is not there raises ReadingStorageError naming the path looked for. Every reading comes back as a dictionary keyed by the header row, every value as text. Returns a list of dictionaries.

RUN COMMAND: None. metermind/data_access/file_storage.py gets a demonstration on Week 1 Day 5, once the loader that reads the register is written, and metermind/core/tariff_calculations.py gets one on Week 2 Day 1, once proration completes it.

GENERATED FILES: None. This day writes only where a caller asks it to.

WEEK 1 DAY 4 TASK

The class of Week 1 Day 2 becomes a hierarchy. One base class holds everything common to a connection, two subclasses override only what their category bills differently, and a factory returns the right one from a record.

MODULE: metermind/core/entities.py

Three classes and one factory. Nothing outside this module decides which class to build; callers hand a record to the factory and are given a connection that already knows how it is billed.

CONSTANTS FROM config/constants.py: CONSUMER_CATEGORIES (tuple, the three categories billed), HOURS_IN_A_DAY, CAPACITY_ALLOWANCE_FACTOR (what a connection could physically draw is derived from these two together with sanctioned load and cycle length), SLAB_TABLE_INR_BY_CATEGORY, FIXED_CHARGE_INR_PER_KW_BY_CATEGORY, FIXED_CHARGE_INR_PER_HORSEPOWER, HORSEPOWER_PER_KILOWATT, ELECTRICITY_DUTY_PERCENT_BY_CATEGORY, SUBSIDY_INR_PER_UNIT_BY_CATEGORY, SUBSIDY_QUALIFYING_UNITS_BY_CATEGORY, MINIMUM_CYCLE_DAYS, MAXIMUM_CYCLE_DAYS.

CLASS: ServiceConnection

One connection, held as an object, able to price itself and to state why it would be refused. This is the domestic connection and it is also the base every other category inherits from. Nine members in all: a constructor and eight methods.

CONSTRUCTOR __init__(self, account_number, consumer_category, sanctioned_load_kw, cycle_days, previous_reading, current_reading):

- A category that is not one of CONSUMER_CATEGORIES raises UnsupportedCategoryError naming the category. An object that cannot be billed must not be built at all.
- The six values are held as attributes under the names they were given.
- physical_maximum_units is derived here and held as an attribute. Both the rollover reading and one of the refusals depend on it, so it is settled once.
- units_consumed is derived here and held as an attribute, by calling calculate_units_consumed of Week 1 Day 3. The arithmetic is not repeated in this module.

METHOD applicable_slab_table(self): Returns the block table this connection's category is billed against, read from SLAB_TABLE_INR_BY_CATEGORY. Every method that needs the table asks this one for it, so a subclass that must be billed against a different table overrides one method rather than several.

METHOD energy_charge_inr(self): Returns the energy charge for this connection's consumption. The base class charges TELESCOPICALLY. The table charged against is whatever applicable_slab_table returns.

METHOD fixed_charge_inr(self): Returns the fixed charge this connection carries. The base class bills for each kilowatt sanctioned, at the category's published rate. The charge follows the sanctioned load and not the consumption.

METHOD electricity_duty_inr(self): Returns the electricity duty this connection carries — the category's published percentage of the energy charge alone, obtained by calling energy_charge_inr rather than repeating it, so a subclass that changes how energy is charged changes the duty without touching this method.

METHOD subsidy_inr(self): Returns the rebate this connection earns. The base class means-tests the rebate by consumption: the whole consumption earns it at or under the category's qualifying figure, and nothing earns it above.

METHOD net_bill_inr(self): Returns what this connection's cycle costs. The four components are obtained by calling the four methods above. A bill is never negative — the agricultural rebate exceeds the agricultural tariff, so a small agricultural cycle reaches zero and stops there.

METHOD refusal_reasons(self): Returns every reason this reading would be refused, and an empty list where there is none. Three rules are checked, and all three are checked every time. A reading breaking two rules reports two reasons. A meter that went backwards with no rollover explaining it is one reason; consumption above what the sanctioned load could physically draw is a second; a billing cycle shorter or longer than a cycle may run is a third. Each reason is a sentence readable on its own. These are the sentences the service returns to a caller on Week 2 Day 4. The cleaning pipeline of Week 2 Day 1 publishes its own set in REJECTION_REASONS: it judges a raw book row rather than a built connection, so it can refuse for things a connection could never be built from at all, and its wording is its own.

METHOD is_billable(self): States whether this reading may be billed. A reading is billable exactly when refusal_reasons is empty; this method is derived from that one rather than repeating its checks.

CLASS: CommercialConnection(ServiceConnection)

A commercial connection, which differs from the base in one respect only: it overrides energy_charge_inr alone.

METHOD energy_charge_inr(self): Returns the energy charge under the NON-TELESCOPIC convention this category is billed by. The whole consumption is charged at the single rate of the block it lands in, not block by block.

CLASS: AgriculturalConnection(ServiceConnection)

An agricultural connection, which differs from the base in two respects: it overrides fixed_charge_inr and subsidy_inr.

METHOD fixed_charge_inr(self): Returns the fixed charge, billed by horsepower rather than by kilowatt. The sanctioned load is converted to horsepower and charged at the published rate for each one.

METHOD subsidy_inr(self): Returns the rebate, earned on every unit whatever it used — there is no qualifying consumption to test against for this category.

FUNCTION: create_service_connection(record)

PURPOSE: Returns a connection of the class the record's category calls for, already built.

REQUIREMENTS:

- The category is settled to lower case with surrounding whitespace removed before it is read, because the book carries it spelled several ways.
- Each of the three categories maps to its own class. A category outside the three raises UnsupportedCategoryError naming it.
- The account number is settled to upper case with surrounding whitespace removed.
- Every numeric field is converted to its type whatever the record supplied it as.
- No caller outside this module names a connection class. This function is how one is obtained.

RETURNS: A ServiceConnection, a CommercialConnection or an AgriculturalConnection.

MAIN BLOCK: Prices the same cycle under all three categories and prints one line each, then states why the agricultural bill is nil.

RUN COMMAND:

```
python3 -m metermind.core.entities
```

CONSOLE OUTPUT:

```
domestic      467 units on a 3 kW connection -> Rs. 2854.48
commercial    467 units on a 3 kW connection -> Rs. 4812.66
agricultural  467 units on a 3 kW connection -> Rs. 0.0
an agricultural cycle costs nothing because the rebate of Rs. 2.6 a unit is larger than the tariff of Rs. 2.25 a unit, and a bill is
never negative
```

GENERATED FILES: None.

WEEK 1 DAY 5 TASK

Three modules gain work today: two wrappers that guard a call and time it, the identifier checks the whole platform validates with, the proration a mid-cycle tariff change forces, and a loader that turns rows into connection objects.

MODULE: metermind/core/decorators.py

Two decorators. Neither changes what the function it wraps returns.

FUNCTION: reject_negative_readings(wrapped_function)

PURPOSE: Returns a wrapper that refuses a call carrying a negative number before the wrapped function runs.

REQUIREMENTS:

- Every argument is examined, whether it was passed by position or by name.
- A negative number raises InvalidReadingError naming the wrapped function and the value. The wrapped function is not called at all.
- A boolean is not a number for this purpose, even though the language counts it as one. A call carrying False must not be refused.
- The wrapper carries the wrapped function's name and documentation forward.
- A call carrying nothing negative returns exactly what the wrapped function returned.

FUNCTION: measure_execution_duration(wrapped_function)

PURPOSE: Returns a wrapper that records how long the wrapped function took without changing what it returned.

REQUIREMENTS:

- The elapsed time of the most recent call is published on the wrapper itself as last_duration_seconds.
- last_duration_seconds exists and is zero before the wrapper has ever been called.
- The wrapper returns exactly what the wrapped function returned — not the duration, and not a pair.
- The wrapper carries the wrapped function's name and documentation forward.

MAIN BLOCK: Wraps a small function, calls it, and prints both what it returned and what the wrapper timed.

MODULE: metermind/core/validators.py

Four checks, each answering a plain yes or no. The service of Weeks 2 and 3, the cleaning of Week 2 Day 1 and the reference extraction of Week 6 Day 3 all validate through these.

CONSTANTS: ACCOUNT_NUMBER_PATTERN, METER_SERIAL_PATTERN, BILLING_PERIOD_PATTERN, EMAIL_ADDRESS_PATTERN – the published shape of each.

FUNCTION: validate_account_number(account_number)

States whether the value is a well-formed account number. The value must match the whole of the published pattern, not merely contain something matching it. Surrounding whitespace is disregarded. A value that is not text returns False rather than raising. Returns a boolean.

FUNCTION: validate_meter_serial(meter_serial)

As validate_account_number, against METER_SERIAL_PATTERN.

FUNCTION: validate_billing_period(billing_period)

As validate_account_number, against BILLING_PERIOD_PATTERN. A month outside the twelve is not well formed.

FUNCTION: validate_email_address(email_address)

As validate_account_number, against EMAIL_ADDRESS_PATTERN, with the value settled to lower case before the check.

MAIN BLOCK: Prints one line for a well-formed and a malformed account number, and the same for a billing period.

MODULE: metermind/core/tariff_calculations.py (continued)

Gains two functions today. A tariff order may be replaced part way through a billing cycle, and a cycle spanning two orders is billed against both. Week 2 Day 1 adds a third, which prices a whole book at once.

CONSTANTS: UNLIMITED_UNITS, CONSUMER_CATEGORIES.

FUNCTION: calculate_prorated_slab_limits(blocks, day_share)

PURPOSE: Returns the same block table with its limits scaled to the share of the cycle it governs.

REQUIREMENTS:

- Every block limit is scaled by the share. A block allowing a hundred units over a whole cycle allows fewer over part of one.
- The rates are never scaled. A rate is a rate whatever part of the cycle it applies to.
- The block that has no upper limit keeps the limit standing for that, unscaled.

FUNCTION: calculate_prorated_energy_charge(consumer_category, units, cycle_days, days_on_the_earlier_order, earlier_blocks, later_blocks)

PURPOSE: Returns the energy charge for a cycle that spans a change of tariff order.

REQUIREMENTS:

- The cycle is split by days, and the consumption is apportioned to each part in the same proportion.
- Each part is charged against its own order's table, with that table's limits scaled to that part's share by calculate_prorated_slab_limits.
- The two parts are added.
- A cycle whose two orders carry the same table must come out at exactly what the whole cycle would have cost on that table alone. This is what makes the proration correct rather than merely plausible.

RETURNS: A float, the energy charge in rupees.

MODULE: metermind/data_access/file_storage.py (continued)

METHOD: load_connection_objects(self, file_name)

PURPOSE: Reads a reading book and returns it as connection objects rather than as dictionaries.

REQUIREMENTS:

- The rows are obtained by calling load_readings_csv, and each is turned into a connection by create_service_connection of Week 1 Day 4. Neither is reimplemented here.
- A row the factory will not accept — an unsupported category, a missing column, a figure that will not convert — is passed over and the rest of the book is still returned. One bad row in a file of five thousand must not cost the caller the other four thousand nine hundred and ninety-nine.
- The order of the file is kept.

RETURNS: A list of connection objects.

MAIN BLOCK: Loads data/reading_book_predict.csv through this method and prints how many connections were built from it.

> **INPUT:** data/reading_book_predict.csv, read by this demonstration.

RUN COMMAND:

```
python3 -m metermind.core.validators
python3 -m metermind.core.decorators
python3 -m metermind.data_access.file_storage
```

CONSOLE OUTPUT:

```
account number ACC10000123    well formed: True
account number ACC1           well formed: False
billing period 2026-06        well formed: True
billing period 2026-13        well formed: False
add_up([1, 2, 3]) returned 6 and the wrapper timed it at 0.000001 seconds
connections loaded: 5000
```

GENERATED FILES: None.

WEEK 2 DAY 1 TASK

The whole book is priced in one pass rather than a row at a time, and made fit to be priced first: every row that cannot be true is refused, and the reason it was refused is recorded beside it.

MODULE: metermind/core/tariff_calculations.py (continued)

FUNCTION: calculate_bill_amounts(book)

PARAMETERS: book – DataFrame. A reading book carrying at least units_consumed, sanctioned_load_kw and consumer_category.

PURPOSE: Prices a whole reading book at once and returns the four charges and the net amount as columns.

REQUIREMENTS:

- The whole book is priced without looping over its rows. Thirty thousand rows priced one at a time is the wrong tool for the job.
- Each category is priced under its own convention, telescopic or otherwise, and its own fixed charge basis, kilowatt or horsepower.
- The rebate is means-tested by consumption for each category as elsewhere in the platform.
- The net amount is the energy charge plus the fixed charge plus the duty, less the rebate, and never below zero.
- A row must come out at the same figure this book would have produced one reading at a time through calculate_bill_amount. Two prices for one reading is a platform that cannot be trusted.

RETURNS: A Series named net_bill_inr, one value per reading in the order supplied, carrying the same index the book was handed with.

MODULE: metermind/data_access/cleaning_pipeline.py

Three published sequences and three functions. Everything the models of Weeks 3 to 5 are fitted on comes out of here, so a row this module keeps is a row those weeks trust.

CONSTANTS: CONSUMER_CATEGORIES, MINIMUM_CYCLE_DAYS, MAXIMUM_CYCLE_DAYS, HOURS_IN_A_DAY, IMPLAUSIBLE_CAPACITY_FACTOR (the multiple of the sanctioned load above which a consumption is not merely suspicious but impossible — a different question from the capacity allowance of Week 1 Day 4: a reading a little above the sanctioned load is flagged for somebody to look at, and a reading far above it is refused outright), METER_ROLLOVER_UNITS, READING_BOOK_TRAIN_PATH, CLEAN_READING_BOOK_PATH, REJECTED_READING_BOOK_PATH.

PUBLISHED NAME: REJECTION_REASONS – Tuple of text. The eight reasons a row may be refused, in the order they are tested: a required field is empty; the read date could not be read; the billing period is not a year and a month; the consumer category has no tariff schedule; the sanctioned load is not above zero; the billing cycle length is outside the permitted range; the current reading is below the previous reading; the consumption could not be drawn by any load on file. A row is never refused for a reason outside this tuple.

PUBLISHED NAME: REQUIRED_FIELDS – Tuple of text. The nine columns a row must carry a value in before any other rule is applied.

PUBLISHED NAME: READ_DATE_FORMATS – Tuple of text. The four date shapes the raw book carries, tried in this order: %Y-%m-%d, %d/%m/%Y, %d-%b-%Y, %d.%m.%Y.

FUNCTION: parse_read_dates(book)

PURPOSE: Returns the read date of every row as a real date, whichever of the four shapes it was written in.

REQUIREMENTS:

- All four published shapes are read, in order, until one succeeds; rows already read are not attempted again.
- A row whose date matches none of the four comes back empty rather than raising.
- The result carries one entry per row, in order, keeping the book's own row labels.

RETURNS: A Series of dates, empty where the date could not be read.

FUNCTION: first_broken_rule(row)

PARAMETERS: row – Series. One row of the reading book, carrying the parsed read date in a column named read_on.

PURPOSE: Returns the first published reason this row is refused, and an empty string where it is refused for none.

REQUIREMENTS:

- The rules are applied IN THE PUBLISHED ORDER, and the first one broken is the answer.
- A row missing a value in any of REQUIRED_FIELDS is refused before any other rule is tried.
- The remaining rules test, in order: a read date that could not be read, a billing period that is not a year and a month, a consumer category with no tariff schedule, a sanctioned load that is not above zero, a cycle length outside the permitted range, a meter that went backwards without a rollover explaining it, and a consumption no load on file could have drawn. A backwards meter is explained by a rollover only where the connection's physical maximum at IMPLAUSIBLE_CAPACITY_FACTOR times its sanctioned load would have carried it past METER_ROLLOVER_UNITS.
- Every reason returned is a member of REJECTION_REASONS, unaltered.

RETURNS: Text, one of REJECTION_REASONS, or an empty string.

FUNCTION: clean_reading_book(book_path, clean_path, rejected_path)

PURPOSE: Cleans a whole reading book, writes both outcomes, and reports how many went each way.

REQUIREMENTS:

- Every numeric column is converted before any rule is tested; a value that will not convert becomes empty rather than raising.
- The read date is parsed through parse_read_dates and each row judged by first_broken_rule.
- NO ROW IS LOST. The rows written to the two files add back exactly to the rows read.
- The rejected file carries the reason against each row in a column named rejection_reason; the clean file does not carry that column at all.
- Both parent folders are created if they are not there.

RETURNS: A dictionary of two counts, keyed kept and rejected.

MAIN BLOCK: Cleans data/reading_book_train.csv into the two files below and prints the two counts.

> **INPUT:** data/reading_book_train.csv – the raw reading book, 30,000 rows.

RUN COMMAND:

```
python3 -m metermind.core.tariff_calculations
python3 -m metermind.data_access.cleaning_pipeline
```

CONSOLE OUTPUT:

```
units consumed: 467.0
energy charge : 2575.7
current demand: 2854.48
a cycle split 14 days on the old order and 16 on the new: 2675.33

kept: 28021 rejected: 1979
```

GENERATED FILES: processed_data/readings_clean.csv (28,021 rows — every model day from Week 3 Day 5 onwards is fitted on this file); processed_data/readings_rejected.csv (1,979 rows, each with the published reason).

WEEK 2 DAY 2 TASK

The clean book moves off the file system and into a database, and the billing office gets the three grouped reports it reads.

MODULE: metermind/data_access/database_loader.py

One published name and one class of four members. The table this module builds is what the reporting of Week 2 Day 2, the look_up_reading tool of Week 7 Day 4 and the assistant of Week 8 all read.

CONSTANTS: CLEAN_READING_BOOK_PATH, READINGS_DATABASE_PATH.

PUBLISHED NAME: READINGS_TABLE – Text. The name of the table this module owns. Every statement in the module names the table through this rather than spelling it.

CLASS: ReadingDatabase

A connection to one SQLite file, holding the reading book. Four members: a constructor and three methods.

CONSTRUCTOR __init__(self, database_path): Opens the database (created if not there), held as an attribute named connection. Rows are returned in a form readable by column name, not only by position.

METHOD create_readings_table(self, book): Builds the readings table from the book's own columns and loads every row into it. reading_id is the primary key. Any table left from an earlier load is dropped first. Every row is loaded and committed. Returns how many rows were loaded.

METHOD find_reading_by_identifier(self, reading_identifier): Returns one reading by its identifier. An identifier nothing is stored against returns nothing rather than raising — the look_up_reading tool of Week 7 Day 4 is the caller that turns it into an error. The identifier is bound as a parameter of the statement and never assembled into its text.

METHOD delete_reading_by_identifier(self, reading_identifier): Removes one reading and reports that it removed it. An identifier nothing is stored against raises ReadingStorageError naming it. The removal is committed before the method returns.

MAIN BLOCK: Loads processed_data/readings_clean.csv into the database and prints how many rows were stored.

MODULE: metermind/data_access/database_reports.py

Three published queries, three functions that run them and one that writes them out, in SQL rather than in Python.

CONSTANTS: READINGS_DATABASE_PATH, READING_REPORTS_PATH.

PUBLISHED NAMES: EXCEPTION_RATE_QUERY, ESTIMATED_READ_QUERY, BILLED_UNITS_QUERY – three SQL statements, each grouping the readings table one way.

FUNCTION: exception_rate_by_category(database)

Returns, for each consumer category, how many cycles were flagged and what share need somebody to go and look, via EXCEPTION_RATE_QUERY. Returns a list of dictionaries, keyed by column name, one row per category present.

FUNCTION: estimated_read_share_by_month(database)

As above, through ESTIMATED_READ_QUERY, grouped by billing period.

FUNCTION: billed_units_by_division(database)

As above, through BILLED_UNITS_QUERY, grouped by division. The units column is stored as text and must be read as a number before it is summed or averaged.

FUNCTION: write_report_files(database, report_path)

Runs all three reports and writes them into one file. Each report is written under a heading in capitals, followed by its column names, followed by its rows, followed by a blank line. The parent folder is created if not there. Returns the path written.

MAIN BLOCK: Writes all three reports to output/reading_reports.csv and prints where it wrote them.

> **INPUT:** processed_data/readings_clean.csv – written by Week 2 Day 1. This day cannot run before it.

RUN COMMAND:

```
python3 -m metermind.data_access.database_loader
python3 -m metermind.data_access.database_reports
```

CONSOLE OUTPUT:

```
stored: 28021
```

```
written: output/reading_reports.csv
```

GENERATED FILES: database/metermind.sqlite (read again by Week 7 Day 4 and the assistant of Week 8); output/reading_reports.csv.

WEEK 2 DAY 3 TASK

The domain becomes a service. One health route today, and a request model that refuses a malformed body before any handler sees it.

MODULE: metermind/api/__init__.py and metermind/api/routers/__init__.py – package markers, empty.

MODULE: metermind/api/schemas.py

The models every route validates against. Two today; grows on Week 2 Day 4, Week 3 Days 2–4 and Week 5 Day 1.

CONSTANTS: MINIMUM_CYCLE_DAYS, MAXIMUM_CYCLE_DAYS (enforced at the door), METER_ROLLOVER_UNITS.

CLASS: HealthReply(BaseModel)

Fields: service (text), status (text). Both required; the health route declares this as its response model.

CLASS: ReadingSubmission(BaseModel)

The body a caller submits a reading with. Nine fields: account_number, meter_serial, consumer_category, billing_period, sanctioned_load_kw (float, above zero, not above five hundred), cycle_days (integer, within published bounds), previous_reading, current_reading (float, not below zero, below the meter's ceiling), read_type (defaults to "actual"). The four numeric bounds are declared on the fields themselves, so a body breaking one is refused with 422 before any handler runs. It is the parent of the screening request of Week 5 Day 1.

VALIDATOR account_number_is_well_formed(cls, given): The check is validate_account_number of Week 1 Day 5, not restated. A malformed value raises with a message stating the shape expected. A well-formed value is returned settled to upper case with surrounding whitespace removed.

VALIDATOR meter_serial_is_well_formed(cls, given): As above, through validate_meter_serial.

VALIDATOR billing_period_is_well_formed(cls, given): As above, through validate_billing_period. A month outside the twelve is refused.

MODULE: metermind/api/application_factory.py

Two published names, one route handler and the factory. Grows on Week 2 Day 4, Week 3 Days 2–4, Week 5 Day 1 and Week 8 Day 3.

PUBLISHED NAMES: API_PREFIX, SERVICE_NAME – the prefix every route is mounted under, and what the service calls itself.

FUNCTION: read_health()

Answers that the service is up. Returns a HealthReply. Takes nothing, reads nothing, needs no token.

FUNCTION: build_application()

REQUIREMENTS:

- A NEW application is built on every call. Two calls must not share anything.
- The health route is mounted under API_PREFIX, answers GET, declares HealthReply as its response model.
- EVERY route the platform ever adds is mounted under API_PREFIX.

MAIN BLOCK: Serves the application on the local host.

RUN COMMAND:

```
python3 -m uvicorn "metermind.api.application_factory:build_application" --factory --port 8000
```

Then, in a second terminal:

```
curl -s http://127.0.0.1:8000/api/v1/health
```

CONSOLE OUTPUT:

```
{"service":"metermind","status":"up"}
```

The interactive documentation page at http://127.0.0.1:8000/docs lists every route added so far, and every route added from here on appears on it without anything being written to put it there.

GENERATED FILES: None.

WEEK 2 DAY 4 TASK

The service gains its first two reading routes, and a store to keep readings in. The pricing behind them is Week 1 Day 4's, called rather than repeated.

MODULE: metermind/api/repository.py

CLASS: InMemoryReadingRepository

Keeps readings for as long as the process lives. Four members today: a constructor and three methods.

CONSTRUCTOR __init__(self): Starts an empty store that can hand out identifiers. Each store counts its own identifiers.

METHOD add_reading(self, reading): Stores a reading, gives it an identifier and returns it as stored. The store assigns the identifier — a caller does not supply one. Identifiers are issued in sequence and never reused within a store. What is returned is a COPY.

METHOD get_reading(self, reading_identifier): Returns one stored reading. An identifier the store does not hold raises ReadingStorageError naming it. Returns a copy.

METHOD list_readings(self): Returns every stored reading, each as a copy, in a new list.

MODULE: metermind/api/schemas.py (continued)

CLASS: ReadingReply(BaseModel)

Fields: reading_id, account_number, consumer_category, units_consumed, energy_charge_inr, fixed_charge_inr, net_bill_inr, is_billable, refusal_reasons (list of text, empty where billable). A refused reading is answered with this same model, carrying its reasons, rather than with an error.

MODULE: metermind/api/routers/readings.py

The router, its store, one helper the module keeps to itself, and two routes today. Four more arrive on Week 2 Day 5, Week 3 Day 2 and Week 3 Day 4.

PUBLISHED NAMES: router, reading_repository – reading_repository is a module-level name so build_application can replace it on Week 3 Day 2 without any route being edited.

FUNCTION: price_reading(stored_reading)

Turns a stored reading into the reply the routes answer with. The connection is built by create_service_connection of Week 1 Day 4 and every figure is read off it — NO ARITHMETIC IS DONE HERE. Units rounded to one decimal place, rupee figures to two. Both the decision and the reasons are carried, whether or not billable. Returns a ReadingReply.

ROUTE: POST /readings — submit_reading(submitted_reading: ReadingSubmission)

REQUIREMENTS:

- Answers 201 on success.
- A body that fails the model is refused with 422 before this handler is entered.
- A reading the domain will not accept at all is refused with 422 carrying the domain's own message, and NOT STORED.
- A reading the domain accepts but will not bill IS STORED and answered with its reasons: unbillable-but-true is kept, because the exception reporting of Week 5 Day 2 is built out of exactly those.

ROUTE: GET /readings/{reading_identifier} — read_reading(reading_identifier: str)

An identifier the store does not hold is answered 404 carrying the store's own message. Built by price_reading.

> **RUN COMMAND (with the service running, second terminal):**

```
curl -s -X POST http://127.0.0.1:8000/api/v1/readings \
  -H "Content-Type: application/json" \
  -d '{"account_number":"ACC10000123","meter_serial":"MTR-NS-004411",
       "consumer_category":"domestic","billing_period":"2026-06","sanctioned_load_kw":3,
       "cycle_days":30,"previous_reading":14208,"current_reading":14675}'
```

CONSOLE OUTPUT:

```
{"reading_id":"RDG40000001","account_number":"ACC10000123","consumer_category":"domestic",
 "units_consumed":467.0,"energy_charge_inr":2575.7,"fixed_charge_inr":150.0,
 "net_bill_inr":2854.48,"is_billable":true,"refusal_reasons":[]}
```

Fetching RDG00000000, which nothing stored, answers 404.

GENERATED FILES: None. This store keeps readings in memory only, gone when the service stops. Week 3 Day 1 gives the service a store that survives a restart.

WEEK 2 DAY 5 TASK

Two mapped tables and a session factory, so the platform has an object-relational mapping to put behind the service on Week 3 Day 1. The store and routes complete their four operations.

MODULE: metermind/api/orm_models.py

PUBLISHED NAME: Base – the declarative base every mapped class inherits from.

CLASS: ConnectionRow(Base)

Table "connections". Fields: account_number (primary key), consumer_name, consumer_category, tariff_code (required), sanctioned_load_kw (float, required), meter_serial (required, unique across the table), division, subdivision (required).

CLASS: ReadingRow(Base)

Table "readings". Fields: reading_id (primary key), account_number (required, indexed), meter_serial, consumer_category (required), billing_period (required, indexed), sanctioned_load_kw (required), cycle_days (required), previous_reading, current_reading (required), read_type (required, default "actual"), case_status (optional — empty until Week 3 Day 3), is_billable (optional).

FUNCTION: build_session_factory(database_path)

REQUIREMENTS:

- BOTH tables are created, not only the one the caller happens to want.
- Creating tables that already exist is not an error.
- Each call returns its own factory; two sessions from it are independent.
- An object read through a session remains usable after that session is closed.

MAIN BLOCK: Builds the factory against the platform's database and prints the tables that now exist.

MODULE: metermind/api/repository.py (continued)

METHOD: replace_reading(self, reading_identifier, reading)

Replaces a stored reading and returns it as stored. An identifier the store does not hold raises ReadingStorageError. The identifier is KEPT.

METHOD: remove_reading(self, reading_identifier)

Removes a stored reading. An identifier the store does not hold raises ReadingStorageError, so removing the same reading twice fails the second time.

MODULE: metermind/api/routers/readings.py (continued)

ROUTE: PUT /readings/{reading_identifier}

An identifier the store does not hold is answered 404. A body the domain will not accept is answered 422. The reply is the replacement priced, not the original.

ROUTE: DELETE /readings/{reading_identifier}

Answers 204 with no body. An identifier the store does not hold is answered 404.

RUN COMMAND:

```
python3 -m metermind.api.orm_models
```

CONSOLE OUTPUT:

```
tables: ['connections', 'readings']
```

```
curl -s -X PUT http://127.0.0.1:8000/api/v1/readings/RDG40000001 \
  -H "Content-Type: application/json" \
  -d '{"account_number":"ACC10000123","meter_serial":"MTR-NS-004411",
       "consumer_category":"domestic","billing_period":"2026-06","sanctioned_load_kw":5,
       "cycle_days":30,"previous_reading":14208,"current_reading":14675}'

curl -s -o /dev/null -w "%{http_code}\n" -X DELETE \
  http://127.0.0.1:8000/api/v1/readings/RDG40000001
```

The delete answers 204.

GENERATED FILES: database/metermind_orm.sqlite – a different file from the reporting database of Week 2 Day 2, deliberately: that loader rebuilds its table from the clean book on every run, and a service whose stored readings vanish on a reload is not a store.

WEEK 3

WEEK 3 DAY 1 TASK

A published interface goes above the store, and a second store answers it against the database of Week 2 Day 5. The routes are not edited to make this work, which is the point of the day.

MODULE: metermind/api/repository.py (continued)

PUBLISHED NAME: STORED_READING_FIELDS – Tuple of text. The eleven fields a stored reading carries, whichever store is holding it. Both stores return exactly these.

CLASS: ReadingRepository(abc.ABC)

The interface every store answers. Five operations declared abstract: add_reading, get_reading, replace_reading, list_readings, remove_reading. A class inheriting from this and implementing only some of the five CANNOT BE CREATED AT ALL. The interface holds no state and no logic.

CLASS: InMemoryReadingRepository – edited today. Now declares ReadingRepository as its parent and gains remove_reading. Behaviour unchanged.

MODULE: metermind/api/sql_repository.py

CLASS: SqlReadingRepository(ReadingRepository)

The store that survives a restart. Six members: a constructor and the five operations.

CONSTRUCTOR __init__(self, database_path=READINGS_DATABASE_PATH): Opens the database, creates the tables if not there, via build_session_factory of Week 2 Day 5.

METHOD add_reading(self, reading): Answers exactly as the in-memory store answers: the store assigns the identifier, and what comes back carries STORED_READING_FIELDS and nothing else. Committed before returning. A field the caller supplied that is not a stored field is not written.

METHOD get_reading(self, reading_identifier): An identifier the database does not hold raises ReadingStorageError, with the same message shape the in-memory store uses.

METHOD replace_reading(self, reading_identifier, reading): An identifier the database does not hold raises ReadingStorageError. Identifier kept, every stored field updated, committed.

METHOD list_readings(self): Order settled by identifier, so two calls against an unchanged database answer in the same order.

METHOD remove_reading(self, reading_identifier): An identifier the database does not hold raises ReadingStorageError, committed.

MODULE: metermind/api/routers/readings.py – edited today, no new top-level names. NOTHING IN THE ROUTES CHANGES. The whole day is demonstrated by that: a second store is written, and every route already works against it, because the routes were written against what a store does rather than against which store it is.

RUN COMMAND:

```
python3 -m pytest tests.py -q -k week3_day1
```

GENERATED FILES: None until a caller names a database.

WEEK 3 DAY 2 TASK

The service is wrapped in the layers every request passes through, told which store to keep readings in, and given password hashing for the accounts of Week 3 Day 4.

MODULE: metermind/api/application_factory.py (continued)

FUNCTION: choose_reading_repository(database_path)

Handed nothing, returns the in-memory store. Handed a database to open, returns the SQL store opened against it.

FUNCTION: build_application(database_path="")

Edited today — takes the database to open, hands it to choose_reading_repository, sets the result on the readings router, and adds the three layers below. The store is set on the router's module-level name, so routes reach it without being edited. The rate limiter and the request log are both added, and the cross-origin policy is applied.

MODULE: metermind/api/middleware.py

CONSTANTS: REQUEST_LOG_PATH, REQUESTS_ALLOWED_PER_WINDOW, RATE_LIMIT_WINDOW_SECONDS, ALLOWED_ORIGINS.

PUBLISHED NAME: REQUEST_LOG_FIELDS – five fields, in order: received_at, method, path, status_code, duration_ms.

CLASS: RequestLogMiddleware

CONSTRUCTOR __init__(self, application, log_path=REQUEST_LOG_PATH): The folder the log sits in is created here rather than on the first request.

METHOD dispatch(self, request, call_next): Times the request around the call. The line carries the five published fields, in order, separated. Written AFTER the answer is known. The answer is returned untouched. One request writes exactly one line.

CLASS: FixedWindowRateLimiter

CONSTRUCTOR __init__(self, application, allowed=REQUESTS_ALLOWED_PER_WINDOW, window_seconds=RATE_LIMIT_WINDOW_SECONDS): Both limits are parameters with published defaults.

METHOD dispatch(self, request, call_next): Callers counted separately. The window is FIXED, not rolling. A request past the limit is answered 429 and IS NOT PASSED DOWN.

FUNCTION: add_cross_origin_policy(application)

Only the published origins are allowed. The four methods the service answers are named, credentials permitted. Returns the application.

MAIN BLOCK: Prints the log fields, the rate limit and the origins the policy allows.

MODULE: metermind/api/password_hashing.py

PUBLISHED NAME: hasher – the password hasher, built once at module level and shared.

FUNCTION: hash_password(plain_password)

What is returned NEVER CONTAINS THE PASSWORD, in any form. Hashing the same password twice returns two different hashes (each carries its own salt). Both hashes still verify against that password.

FUNCTION: verify_password(plain_password, hashed_password)

Returns True only for the right password. A wrong password returns False rather than raising, and so does a stored value that is not a hash at all.

MAIN BLOCK: Hashes a password, prints the beginning of the stored form, and verifies the right password and a wrong one.

RUN COMMAND:

```
python3 -m metermind.api.password_hashing
python3 -m metermind.api.middleware
```

To put the service on the SQL store instead of in memory, run the module rather than the factory command:

```
python3 -m metermind.api.application_factory
```

CONSOLE OUTPUT:

```
the stored hash begins: $argon2id$v=19$m=65536,t=3,p=4$... and then a salt that differs every time
```

```
the right password verifies : True
a different one does not    : False
fields written for every request: ('received_at', 'method', 'path', 'status_code', 'duration_ms')
60 requests are allowed in each 60 second window
origins the browser may call from: ('http://localhost:3000', 'https://billing.example.in')
```

GENERATED FILES: output/logs/requests.log – one line per request answered.

WEEK 3 DAY 3 TASK

A case walks a ladder, only an inspector may move it, and every move is written down.

MODULE: metermind/api/case_transitions.py

CONSTANTS: INITIAL_CASE_STATUS.

PUBLISHED NAME: ALLOWED_TRANSITIONS – Dictionary. All six statuses — raised, assigned, inspected, regularised, billed_back, closed_no_fault — mapped to what they may move to. Three map to nothing (the ends of the ladder).

FUNCTION: move_case_status(current_status, wanted_status)

REQUIREMENTS:

- A status the platform does not know raises MeterMindError naming it.
- A move the ladder does not allow raises MeterMindError stating WHERE THE CASE COULD HAVE GONE INSTEAD, or that it is closed.
- THE LADDER GOES ONE WAY, with one published exception: a case that has been inspected may be sent back to be assigned again.

MAIN BLOCK: Walks two permitted moves and one refused one, printing each.

MODULE: metermind/api/access_control.py

CONSTANTS: TOKEN_SECRET_ENVIRONMENT_NAME, DEVELOPMENT_SECRET, TOKEN_VALID_MINUTES, INSPECTOR_ROLE, AUDIT_TRAIL_PATH.

PUBLISHED NAME: AUDIT_TRAIL_FIELDS – six columns: recorded_at, actor, role, reading_id, from_status, to_status.

PUBLISHED NAME: bearer_scheme – the security scheme guarded routes depend on; declared not to raise on its own, so a missing token reaches the platform's own handling.

FUNCTION: read_signing_secret()

Environment read first, published development secret used where none is set. Both signing and reading go through this one function.

FUNCTION: issue_token(subject, role)

Token carries subject, role, and an expiry set the published number of minutes ahead, computed against a timezone-aware clock. Signed.

FUNCTION: read_token(token)

A token that cannot be read raises 401 with a message saying so and NOTHING MORE — expired, signed elsewhere, altered or absent are all one answer.

FUNCTION: require_inspector(carried=Depends(bearer_scheme))

A request with no token is answered 401. A readable token whose role is not the inspector role is answered 403. 401 AND 403 ARE DIFFERENT ANSWERS TO DIFFERENT QUESTIONS.

FUNCTION: record_audit_entry(actor, role, reading_identifier, from_status, to_status, trail_path=AUDIT_TRAIL_PATH)

The trail is APPENDED TO, never rewritten. Header written only when the file is being created. Time recorded carries its time zone. Parent folder created if not there.

MAIN BLOCK: Issues an inspector token, prints its beginning, and prints what it reads back as.

MODULE: metermind/api/schemas.py (continued)

CLASS: CaseStatusRequest(BaseModel)

Fields: wanted_status, noted_by. Who is moving the case is required, not inferred from the token.

CLASS: CaseStatusReply(BaseModel)

Fields: reading_id, from_status, to_status, recorded_by.

MODULE: metermind/api/routers/readings.py (continued)

ROUTE: PATCH /readings/{reading_identifier}/status

change_case_status(reading_identifier, wanted: CaseStatusRequest, carried=Depends(require_inspector))

REQUIREMENTS:

- The guard is a dependency; a request without an inspector's token never reaches the body.
- A reading the store does not hold is answered 404.
- A case never moved is treated as standing at the published initial status.
- A move the ladder refuses is answered 409, carrying the ladder's own message.
- A permitted move is stored, then written to the audit trail. The trail records what happened, not what was attempted.

RUN COMMAND:

```
python3 -m metermind.api.case_transitions
```

CONSOLE OUTPUT:

```
raised               -> assigned            assigned
assigned             -> inspected           inspected
raised               -> closed_no_fault     refused: a case at raised may move to assigned, not to closed_no_fault
```

GENERATED FILES: output/audit_trail.csv – one line per accepted case move, in the six published fields.

WEEK 3 DAY 4 TASK

Accounts, sign-in, and a listing that needs a token and comes back a page at a time.

MODULE: metermind/api/access_control.py (continued)

FUNCTION: require_signed_in(carried=Depends(bearer_scheme))

A request with no token, or one that cannot be read, is answered 401. A readable token is admitted whatever role it carries.

MODULE: metermind/api/schemas.py (continued)

CLASS: PagedReadings(BaseModel)

Fields: readings (list of ReadingReply), page, page_size, total. total counts what the filters matched, NOT what this page holds.

MODULE: metermind/api/routers/user_accounts.py

CONSTANTS: CONSUMER_ROLE.

PUBLISHED NAMES: router, accounts – accounts is a module-level account store keyed by account number.

CLASS: AccountRequest(BaseModel)

Fields: account_number, email_address, password, role (defaults to consumer role).

VALIDATOR account_number_is_well_formed: as Week 2 Day 3.

VALIDATOR account_email_address_is_well_formed: through validate_email_address; well-formed address returned settled to lower case.

CLASS: AccountReply(BaseModel)

Fields: account_number, email_address, role. THE PASSWORD IS NOT A FIELD, in any form.

CLASS: LoginRequest(BaseModel)

Fields: account_number, password.

CLASS: TokenReply(BaseModel)

Fields: access_token, token_type, role.

ROUTE: POST /accounts — sign_up(wanted: AccountRequest)

Answers 201 on success. An account number already signed up is answered 409 and THE EXISTING ACCOUNT IS NOT OVERWRITTEN. What is stored is the hash, never the password.

ROUTE: POST /accounts/login — log_in(wanted: LoginRequest)

Checked with verify_password. An unknown account number and a wrong password are answered THE SAME WAY, with 401 and one message. A correct sign-in answers with a token carrying the account's role.

MODULE: metermind/api/routers/readings.py (continued)

ROUTE: GET /readings

list_readings(consumer_category, read_type, page=1, page_size=20, carried=Depends(require_signed_in))

REQUIREMENTS:

- The route needs a token of any role.
- The bounds on paging are declared on the parameters (422 before the handler runs for an out-of-range page_size).
- A filter not given narrows nothing; two filters given are both applied.
- total counts what the filters matched, before the page was cut out of them.
- Two consecutive pages hold no reading in common, and together hold every reading the filters matched.
- Every reading on the page is priced by price_reading.

RUN COMMAND:

```
python3 -m metermind.api.access_control
```

```
curl -s -X POST http://127.0.0.1:8000/api/v1/accounts \
  -H "Content-Type: application/json" \
  -d '{"account_number":"ACC10000123","email_address":"meena@example.in",
       "password":"Chennai#2026"}'

curl -s -X POST http://127.0.0.1:8000/api/v1/accounts/login \
  -H "Content-Type: application/json" \
  -d '{"account_number":"ACC10000123","password":"Chennai#2026"}'
```

CONSOLE OUTPUT:

```
issued: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...
reads back as: {'sub': 'je/1180', 'role': 'inspector', 'exp': 1786648972}
{"account_number":"ACC10000123","email_address":"meena@example.in","role":"consumer"}
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...","token_type":"bearer","role":"consumer"}
```

Listing without a token answers 401; with one, 200.

GENERATED FILES: None. Accounts live for as long as the service runs.

WEEK 3 DAY 5 TASK

The first model. One preparation pipeline every model in the platform is fitted through, a baseline to beat, and the split every later comparison is made on.

MODULE: metermind/exception_models/__init__.py – package marker, empty.

MODULE: metermind/exception_models/feature_pipeline.py

Four published names and one function. EVERY model in Weeks 3 to 5 is fitted through this one preparation, which is what makes two of them comparable at all.

PUBLISHED NAMES: MODELLING_NUMBER_COLUMNS (tuple, the ten columns treated as numbers), MODELLING_CATEGORY_COLUMNS (tuple, the five columns treated as categories), MODELLING_COLUMNS (the two above together, derived rather than listed again), EXCEPTION_COLUMN (the column every model learns).

FUNCTION: build_feature_pipeline()

REQUIREMENTS:

- Numeric columns have missing values filled and are then rescaled.
- Categoric columns are widened into indicator columns.
- A category NEVER SEEN WHILE FITTING must not raise when met later.
- A category too rare to be worth its own column is grouped rather than given one.
- The preparation is returned UNFITTED.
- Returned as one object, so a model carries its preparation with it.

MAIN BLOCK: Prepares two thousand rows and prints how many columns went in and how many numbers came out.

MODULE: metermind/exception_models/baseline_model.py

One published name and six functions. The baseline every later family is measured against.

CONSTANTS: EXCEPTION_CLASSES, HELD_OUT_SHARE, MODEL_SEED, CLEAN_READING_BOOK_PATH, BASELINE_MODEL_PATH, BASELINE_SCORES_PATH.

PUBLISHED NAME: EXCEPTION_CLASSES – Tuple of text, the five classes the platform publishes.

FUNCTION: split_readings(book)

REQUIREMENTS:

- The split is STRATIFIED by the exception class.
- Settled by the published seed, so asking twice gives the same rows.
- A row is on one side or the other and never on both; the two sides together are the whole book.
- Only the modelling columns are carried into the fitting rows; the class column is carried separately.

RETURNS: Four objects: fitting columns, held-out columns, fitting classes, held-out classes.

FUNCTION: train_baseline_model(fitting_columns, fitting_classes)

The preparation and the decision are held in ONE OBJECT, with two steps named prepare and decide. Settled by the published seed, enough iterations to settle. Learns ALL FIVE published classes.

FUNCTION: score_exception_model(model, held_out_columns, held_out_classes)

Both an overall accuracy and a per-class report. ACCURACY ALONE IS NOT ENOUGH HERE. A class the model never predicted is scored zero rather than raising.

FUNCTION: save_model(model, model_path)

Parent folder created if not there. What is written is the whole fitted object, preparation included.

FUNCTION: load_model(model_path)

A model saved and reloaded reaches EXACTLY the same decisions on the same rows.

FUNCTION: predict_exception(model, readings)

Only the published modelling columns are handed to the model. Every class returned is one of the five published classes.

MAIN BLOCK: Splits the clean book, fits the baseline, scores it, saves the model and the scores, prints the accuracy.

> **INPUT:** processed_data/readings_clean.csv – written by Week 2 Day 1.

RUN COMMAND:

```
python3 -m metermind.exception_models.feature_pipeline
python3 -m metermind.exception_models.baseline_model
```

CONSOLE OUTPUT:

```
15 columns in, 30 numbers out
accuracy: 0.7991
```

The accuracy moves a little from one machine to another; no testcase pins a score.

GENERATED FILES: artifacts/baseline_exception.joblib; output/baseline_scores.json.

WEEK 4 DAY 1 TASK

Two more families on the same preparation, and the question the accuracy of Week 3 Day 5 does not answer: whether the model can read the classes that matter most.

MODULE: metermind/exception_models/baseline_model.py (continued)

PUBLISHED NAME: FITTING_ROUNDS_LIMIT – Integer. How many rounds the baseline is given to settle.

FUNCTION: build_baseline_model(class_weight=None)

Returns the baseline model UNFITTED, so it can be handed to a cross-validation that will fit it itself. Building and fitting are SEPARATE STEPS from today. The preparation and decision are held in one object, as before.

FUNCTION: split_readings(book, columns=None)

Edited today — handed no columns it splits on the published modelling set, exactly as before. Week 4 Day 3 splits on the widened set through this same function.

FUNCTION: train_baseline_model – edited today, now builds through build_baseline_model rather than assembling the model itself.

MODULE: metermind/exception_models/tree_models.py

Five functions. Two families that read a book differently from the baseline, and the measurement that shows what weighting the classes buys.

CONSTANTS: DECISION_TREE_DEPTH_LIMIT, RANDOM_FOREST_TREE_COUNT, RANDOM_FOREST_DEPTH_LIMIT, MODEL_SEED, CLEAN_READING_BOOK_PATH, DECISION_TREE_MODEL_PATH, RANDOM_FOREST_MODEL_PATH.

FUNCTION: build_decision_tree_model(class_weight=None)

Depth limited to the published figure — a tree grown without a limit memorises the book. Same preparation as every other family. Settled by the published seed.

FUNCTION: build_random_forest_model(class_weight=None)

Published number of trees, each limited to published depth. Fitted on one worker rather than many, so the result does not depend on how many processors the machine has.

FUNCTION: train_decision_tree(fitting_columns, fitting_classes, class_weight=None)

Built through build_decision_tree_model and then fitted.

FUNCTION: train_random_forest(fitting_columns, fitting_classes, class_weight=None)

Built through build_random_forest_model and then fitted.

FUNCTION: compare_class_weighting(fitting_columns, held_out_columns, fitting_classes, held_out_classes)

REQUIREMENTS:

- Both models fitted on the same rows, scored on the same rows — only the weighting differs.
- RECALL IS REPORTED CLASS BY CLASS, not as one number.
- The lowest recall across the five classes is reported for each.
- A class the model never predicted scores zero rather than raising.
- Every one of the five published classes appears in the report, in the published order.

RETURNS: A dictionary of two entries, unweighted and balanced, each holding recall_by_class and lowest_recall.

MAIN BLOCK: Splits the clean book, fits a tree and a forest, saves both, prints each path with its held-back score.

RUN COMMAND:

```
python3 -m metermind.exception_models.tree_models
```

CONSOLE OUTPUT:

```
artifacts/tree_exception.joblib 0.8676
artifacts/forest_exception.joblib 0.884
```

Both figures move a little from one machine to another. The forest reading the hardest class at least as well as the single tree is what the day is judged on.

GENERATED FILES: artifacts/tree_exception.joblib; artifacts/forest_exception.joblib.

WEEK 4 DAY 2 TASK

A fourth family, and a comparison that does not rest on one lucky split.

MODULE: metermind/exception_models/cross_validation.py

CONSTANTS: SUPPORT_VECTOR_REGULARISATION, CROSS_VALIDATION_FOLDS, MODEL_SEED, CLEAN_READING_BOOK_PATH, CROSS_VALIDATION_PATH.

PUBLISHED NAME: CROSS_VALIDATION_FIELDS – model, fold, accuracy.

FUNCTION: build_support_vector_model(class_weight=None)

Linear SVM, unfitted, on the shared preparation, held back by the published regularisation. This family REPORTS NO CONFIDENCE, only a decision — which is why Week 4 Day 4 cannot consider it for champion.

FUNCTION: every_candidate_model(class_weight=None)

All four candidate families, unfitted, keyed by name, returned so a caller may fit each as many times as it needs.

FUNCTION: cross_validate_models(fitting_columns, fitting_classes, folds=None)

REQUIREMENTS:

- Folds are STRATIFIED.
- EVERY FAMILY IS SCORED OVER THE SAME FOLDS.
- Folds settled by the published seed.
- Every family scored the same number of times.
- Fitted on one worker.
- Every fold's score is kept, not only the mean.

RETURNS: A dictionary keyed by family name, each a list of one score per fold.

FUNCTION: fold_spread(fold_scores)

How far apart a family's best and worst folds were. Never negative, never wider than the range the scores themselves span.

FUNCTION: write_cross_validation(scored, where)

Three published columns, header row, ONE ROW PER FAMILY PER FOLD — four families over five folds is twenty rows. Folds numbered from one.

MAIN BLOCK: Cross-validates all four families, writes the file, prints each family's mean and spread.

RUN COMMAND:

```
python3 -m metermind.exception_models.cross_validation
```

About thirty seconds: four families over five folds is twenty fits.

CONSOLE OUTPUT:

```
baseline         mean 0.7236  spread 0.0109
decision_tree    mean 0.8602  spread 0.0105
random_forest    mean 0.8924  spread 0.0074
support_vector   mean 0.7943  spread 0.0158
```

GENERATED FILES: output/cross_validation.csv – twenty rows.

WEEK 4 DAY 3 TASK

Three columns the reading book does not carry, derived from the ones it does, and the measurement that shows whether they earned their place.

MODULE: metermind/exception_models/feature_pipeline.py (continued)

Gains two published names and one function today, and build_feature_pipeline gains a parameter. Every model builder written on Week 3 Day 5, Week 4 Day 1 and Week 4 Day 2 gains the same parameter on the same day, in the same position and with the same default: build_baseline_model, build_decision_tree_model, build_random_forest_model, build_support_vector_model and every_candidate_model all become (class_weight=None, widened=False).

PUBLISHED NAMES: DERIVED_NUMBER_COLUMNS (three columns: consumption_ratio_to_average, load_factor, meter_age_years), WIDENED_MODELLING_COLUMNS (the published modelling columns and the three derived ones).

FUNCTION: modelling_columns(widened=False)

Asking without the derived columns returns exactly what Week 3 Day 5 published.

FUNCTION: build_feature_pipeline(widened=False)

Edited today. Handed widened, prepares the three derived columns as numbers too; handed nothing, behaves exactly as before.

MODULE: metermind/exception_models/feature_engineering.py

CONSTANTS: HOURS_IN_A_DAY, MODEL_SEED, CLEAN_READING_BOOK_PATH, WIDENED_READINGS_PATH, FEATURE_LIFT_PATH, MEASUREMENT_DECIMAL_PLACES.

PUBLISHED NAME: DAYS_IN_A_YEAR – Float, days in a year including the leap quarter.

FUNCTION: add_consumption_ratio_to_average(book)

Adds a column measuring this cycle's consumption against what this connection normally uses. Book handed in is NOT ALTERED; a copy is returned. A cycle equal to the connection's own average comes out at exactly one. A connection with no history to average against is treated as ordinary rather than dropped or left empty. This is the column that separates genuine_high_usage from everything else.

FUNCTION: add_load_factor(book)

Adds a column measuring consumption against what the sanctioned load could physically have drawn. A value ABOVE ONE IS PHYSICALLY IMPOSSIBLE and points at a load on file that is wrong. A very low value points at a meter that has stopped.

FUNCTION: add_meter_age_years(book)

Adds a column holding how old the meter was when the cycle ended. A date that cannot be read leaves the age empty rather than raising.

FUNCTION: widen_readings(book)

Adds all three derived columns by calling the three functions above. EXACTLY THREE COLUMNS ARE ADDED. Result carries WIDENED_MODELLING_COLUMNS among its columns.

FUNCTION: measure_feature_lift(book)

REQUIREMENTS:

- Same family, same seed, same split both times. ONLY THE COLUMN SET DIFFERS.
- Three figures reported: narrow score, widened score, lift.
- All three rounded to published places, and THE LIFT IS THE DIFFERENCE BETWEEN THE TWO ROUNDED FIGURES.

MAIN BLOCK: Widens the whole clean book and writes it, measures the lift on a sample, writes the three figures.

RUN COMMAND:

```
python3 -m metermind.exception_models.feature_engineering
```

CONSOLE OUTPUT:

```
narrow   0.885
widened  0.9137
lift     0.0287
```

GENERATED FILES: processed_data/readings_widened.csv; output/feature_lift.json.

WEEK 4 DAY 4 TASK

Not every mistake costs the same. Calling an honest consumer a thief costs more than missing a meter fault, and today the platform chooses its model and its thresholds on that rather than on accuracy.

MODULE: metermind/exception_models/champion_model.py

CONSTANTS: EXCEPTION_CLASSES, MEASUREMENT_DECIMAL_PLACES, CLEAN_READING_BOOK_PATH, CHAMPION_MODEL_PATH, OPERATING_RULE_PATH.

PUBLISHED NAME: COST_MATRIX – dictionary of dictionaries, what each mistake costs for every true class against each predicted class. Diagonal is zero. NOT SYMMETRIC: calling a genuine high-usage cycle a tamper is the most expensive entry, far dearer than the reverse.

PUBLISHED NAMES: TAMPER_FLOORS, FAULT_FLOORS – two tuples of floats, the confidence floors the search tries.

PUBLISHED NAME: MODELS_THAT_REPORT_CONFIDENCE – the three families that can carry an operating rule.

FUNCTION: choose_champion(fitting_columns, fitting_classes, folds=None)

Scored by cross_validate_models of Week 4 Day 2. ONLY THE FAMILIES THAT REPORT A CONFIDENCE ARE CONSIDERED. Returns the winning family's name, and a dictionary of every considered family's mean.

FUNCTION: expected_cost(truth, predicted)

Each pair charged at the matrix entry for that true class and that prediction, total divided by count. A perfect prediction costs EXACTLY ZERO.

FUNCTION: predict_with_rule(model, columns, tamper_floor, fault_floor)

REQUIREMENTS:

- The model's confidence across all five classes is read.
- Where the leading class is one of the two guarded ones and its confidence is below that class's floor, THE NEXT MOST LIKELY CLASS IS TAKEN INSTEAD.
- A class with no floor stands whatever its confidence.
- Floors of zero must reproduce exactly what the model would have said unaided.

FUNCTION: choose_operating_rule(model, held_out_columns, held_out_classes)

Every combination of published floors tried, each costed by expected_cost. Pair kept is the cheapest. THE RULE CHOSEN IS NEVER DEARER THAN TAKING THE MODEL'S OWN ANSWER. Chosen on rows KEPT BACK.

FUNCTION: save_champion(model, rule, model_path, rule_path)

THE MODEL AND ITS RULE ARE WRITTEN TOGETHER. Every figure rounded to published places.

MAIN BLOCK: Widens the clean book, chooses the champion across folds, fits it, searches floors, saves both, prints the choice and the rule.

RUN COMMAND:

```
python3 -m metermind.exception_models.champion_model
```

CONSOLE OUTPUT:

```
only the families that report a confidence can carry an operating rule, so the support vector
is not considered:
chosen: random_forest {'baseline': 0.7236, 'decision_tree': 0.8602, 'random_forest': 0.8924}
argmax cost: 0.3129
rule: {'tamper_floor': 0.6, 'fault_floor': 0.0, 'expected_cost': 0.3083}
```

GENERATED FILES: artifacts/champion_exception.joblib; output/operating_rule.json.

WEEK 4 DAY 5 TASK

The five thousand waiting readings are classified and sorted into the five queues the company actually works, one file each.

MODULE: metermind/exception_models/investigation_queues.py

CONSTANTS: QUEUE_FILE_PATHS, READING_BOOK_PREDICT_PATH, CHAMPION_MODEL_PATH, OPERATING_RULE_PATH.

PUBLISHED NAMES: QUEUES, QUEUE_BY_DECISION – QUEUES is the queue names taken from the published mapping. QUEUE_BY_DECISION: THERE IS ONE QUEUE FOR EVERY PUBLISHED CLASS AND NO CLASS WITHOUT ONE.

FUNCTION: sort_into_queues(book, decided)

REQUIREMENTS:

- Book not altered; decision attached to a copy.
- EVERY ROW LANDS IN EXACTLY ONE QUEUE. Counts across the five queues add back to rows handed in.
- The queue a reading lands in is the class it was decided to be.
- A queue nothing was sorted into is still present, holding nothing.
- Each queue's rows renumbered from zero.

FUNCTION: queue_counts(sorted_out)

One entry per queue, including empty ones.

FUNCTION: write_queue_files(sorted_out, paths=None)

FIVE FILES ARE WRITTEN, each carrying a header row whether or not it holds readings. Parent folders created if not there.

MAIN BLOCK: Widens the waiting readings, loads the champion and its rule, classifies under the rule, sorts them, writes the five files, prints the counts.

> **INPUT:** data/reading_book_predict.csv (5,000 readings); artifacts/champion_exception.joblib and output/operating_rule.json from Week 4 Day 4. This day cannot run before it.

RUN COMMAND:

```
python3 -m metermind.exception_models.investigation_queues
```

CONSOLE OUTPUT:

```
{'genuine_high_usage': 2342, 'estimated_read': 1253, 'meter_fault': 600, 'tamper_suspected': 273, 'tariff_mismatch': 532}
```

The five counts add to 5,000.

GENERATED FILES: output/queue_no_action.csv (genuine_high_usage — nobody sent); output/queue_meter_reading.csv (estimated_read — a reader sent); output/queue_metering.csv (meter_fault — metering team sent); output/queue_vigilance.csv (tamper_suspected — vigilance sent, most closely guarded by the operating rule); output/queue_billing.csv (tariff_mismatch — billing reviews the sanctioned load on file).

WEEK 5

WEEK 5 DAY 1 TASK

The champion of Week 4 Day 4 goes behind a route, and the two spellings problems the raw book carries are settled at the door rather than inside a model.

MODULE: metermind/api/schemas.py (continued)

CONSTANTS: CONSUMER_CATEGORY_CORRECTIONS – dictionary, the nine spellings the field application produced, mapped to the three the platform bills.

CLASS: ReadingSubmission – edited today. Gains read_date (optional field) and two validators.

VALIDATOR spelling_is_settled_at_the_door(cls, given): Settled to lower case with whitespace removed first. A published misspelling is corrected. A value that is neither a category nor a known misspelling is RETURNED UNCHANGED, not refused here — the domain refuses it later.

VALIDATOR date_is_readable(cls, given): Nothing supplied is accepted (optional field). All four raw shapes are accepted. What is returned is always the ONE shape the platform uses. A value in none of the four shapes is refused.

CLASS: ScreeningRequest(ReadingSubmission)

Everything ReadingSubmission carries, and thirteen more the model needs: units_consumed, twelve_month_average_units, same_month_last_year_units, consecutive_estimated_reads, days_since_actual_read, neighbourhood_average_units, arrears_inr, payment_defaults_last_year, seal_check_result, meter_make, division, meter_install_date, cycle_end_date. IT INHERITS FROM ReadingSubmission rather than restating its fields.

CLASS: ScreeningReply(BaseModel)

Fields: account_number, billing_period, exception_class, confidence, goes_to. The decision is reported with the confidence behind it.

MODULE: metermind/api/routers/screening.py

CONSTANTS: CHAMPION_MODEL_PATH, OPERATING_RULE_PATH, QUEUE_FILE_PATHS.

PUBLISHED NAMES: router, champion_model_path, operating_rule_path, loaded_champion – loaded_champion is empty until the first call, holding model and rule once read.

FUNCTION: load_champion_once()

The model is read ONCE FOR THE LIFE OF THE SERVICE. Model and rule read TOGETHER. A service with no champion fitted answers 503.

ROUTE: POST /screening

screen_reading(submitted_reading: ScreeningRequest, carried=Depends(require_signed_in))

REQUIREMENTS:

- The route needs a token of any role.
- The reading is widened by widen_readings of Week 4 Day 3.
- The class is decided by predict_with_rule, under the saved rule, NOT the model's unaided answer.
- The confidence reported is the confidence IN THE CLASS THAT WAS RETURNED.
- The queue reported is read from the published mapping.

RUN COMMAND (with the service running and a token in hand):

```
curl -s -X POST http://127.0.0.1:8000/api/v1/screening \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"account_number":"ACC10000123","meter_serial":"MTR-NS-004411",
       "consumer_category":"domestic","billing_period":"2026-06","sanctioned_load_kw":5,
       "cycle_days":30,"previous_reading":14208,"current_reading":14675,
       "read_type":"actual","read_date":"2026-07-01","units_consumed":467.0,
       "twelve_month_average_units":180.0,"same_month_last_year_units":195.0,
       "consecutive_estimated_reads":0,"days_since_actual_read":30,
       "neighbourhood_average_units":210.0,"arrears_inr":0.0,
       "payment_defaults_last_year":0,"seal_check_result":"intact","meter_make":"Genus",
       "division":"North","meter_install_date":"2024-08-14","cycle_end_date":"2026-06-30"}'
```

CONSOLE OUTPUT:

```
{"account_number":"ACC10000123","billing_period":"2026-06","exception_class":"meter_fault",
 "confidence":0.5374,"goes_to":"queue_metering.csv"}
```

Without a token the route answers 401.

GENERATED FILES: None.

WEEK 5 DAY 2 TASK

Accuracy is one number and this platform has five classes, two of them rare. Today the champion is reported class by class, with the confusion behind it.

MODULE: metermind/exception_models/exception_report.py

CONSTANTS: CLEAN_READING_BOOK_PATH, CHAMPION_MODEL_PATH, EXCEPTION_REPORT_PATH.

PUBLISHED NAME: EXCEPTION_REPORT_FIELDS – exception_class, precision, recall, support.

FUNCTION: per_class_scores(truth, predicted)

ALL FIVE PUBLISHED CLASSES APPEAR, in published order. A class never predicted scores zero. Precision and recall are different questions and both reported. Supports add back to the rows scored.

FUNCTION: confusion_frame(truth, predicted)

FIVE ROWS BY FIVE COLUMNS, both labelled. Each row totals the support of that class; the diagonal over the total is the accuracy.

FUNCTION: write_exception_report(truth, predicted, where)

Four published columns. ONE ROW PER CLASS AND ONE TOTAL ROW, marked so it can't be read as a sixth class. Every figure rounded to four places. Week 7 Day 4 reads the precision out of this file.

MAIN BLOCK: Scores the champion on rows kept back, prints the confusion table, writes the report.

RUN COMMAND:

```
python3 -m metermind.exception_models.exception_report
```

CONSOLE OUTPUT:

```
                    genuine_high_usage  estimated_read  meter_fault  tamper_suspected  tariff_mismatch
genuine_high_usage                2500              43           26                 9               15
estimated_read                      71            1323            0                 0                0
meter_fault                         69              38          556                65               59
tamper_suspected                     7               0           40               266                0
tariff_mismatch                      0               0            1                 0              517
written: output/exception_report.csv
```

GENERATED FILES: output/exception_report.csv.

WEEK 5 DAY 3 TASK

Attention, written out. Three functions, no library that does it for you.

MODULE: metermind/message_intelligence/__init__.py – package marker, empty.

MODULE: metermind/message_intelligence/attention.py

Three functions. Nothing else in the platform imports them; they exist so the mechanism behind every model of Weeks 6 to 8 is understood rather than taken on faith.

FUNCTION: softmax_rows(scores)

REQUIREMENTS:

- EVERY ROW ADDS TO EXACTLY ONE, and no weight is negative.
- The largest score in each row is taken off before anything is exponentiated (overflow protection; unchanged answer since softmax is shift-invariant).
- A row of scores in the hundreds must come back with finite weights.
- Only the last dimension is turned into weights.

FUNCTION: attention_weights(queries, keys)

REQUIREMENTS:

- Square in the number of positions when queries and keys are the same length, each row adds to one.
- Scores are SCALED BY THE SIZE OF A KEY before softmax — without scaling, long keys produce large scores, weights collapse onto one position.
- Weights produced by softmax_rows, not a second implementation.

FUNCTION: scaled_dot_product_attention(queries, keys, values)

Weights come from attention_weights. WHAT COMES OUT IS SHAPED BY THE VALUES, NOT BY THE KEYS.

MAIN BLOCK: Prints that every row of a softmax adds to one, and the shape three positions attending over three produce.

RUN COMMAND:

```
python3 -m metermind.message_intelligence.attention
```

CONSOLE OUTPUT:

```
every row of a softmax adds to one: [1. 1. 1.]
three positions attending over three, four numbers each -> (3, 4)
```

GENERATED FILES: None.

WEEK 5 DAY 4 TASK

A complaint as received carries a quoted reply, a greeting, a sign-off and a signature block. None of that is the complaint. Today it comes off, and nothing else does.

MODULE: metermind/message_intelligence/text_cleaning.py

Six published patterns and seven functions. Every model that reads a complaint from Week 5 Day 5 onwards reads what comes out of here.

CONSTANTS: COMPLAINT_MESSAGES_PATH, CLEAN_COMPLAINTS_PATH.

PUBLISHED NAMES: QUOTED_REPLY_PATTERN, GREETING_PATTERN, SIGN_OFF_PATTERN, SIGNATURE_PATTERN, COMPLAINT_REFERENCE_PATTERN, WHITESPACE_PATTERN.

FUNCTION: strip_quoted_reply(message)

Quoted lines, forwarded headers and the line introducing a quotation all go, together with what follows. THE NEW MESSAGE STAYS.

FUNCTION: strip_greeting_and_sign_off(message)

Only a line that is WHOLLY a greeting or sign-off goes. A sentence merely containing one of those words stays.

FUNCTION: strip_signature(message)

A line that is wholly a contact number or staff identifier goes. A number quoted inside a sentence stays.

FUNCTION: normalise_whitespace(message)

Line breaks become spaces, runs of spaces collapse to one, ends trimmed.

FUNCTION: clean_message(message)

REQUIREMENTS:

- Every step done by calling the functions above.
- The complaint reference is removed too — DELIBERATE, so a model learns the complaint, not the reference.
- A value that is not text is turned into text rather than raising.
- NO MESSAGE IS EMPTIED.

FUNCTION: clean_messages(messages)

One cleaned message out per message in, in order. Declared at module level (not inside another function), since the classifier pipeline of Week 5 Day 5 carries it as its first step and a pipeline whose first step is a locally-defined function CANNOT BE SAVED AND RELOADED.

FUNCTION: clean_corpus(messages_path, clean_path)

Cleaned text added as a column; THE ORIGINAL IS KEPT. NO MESSAGE IS LOST. Reports messages cleaned and the shortest cleaned message's length.

MAIN BLOCK: Cleans the whole complaint corpus and prints what it did.

> **INPUT:** data/complaint_messages.csv – 5,000 complaints as received.

RUN COMMAND:

```
python3 -m metermind.message_intelligence.text_cleaning
```

CONSOLE OUTPUT:

```
{'messages': 5000, 'shortest': 93}
```

GENERATED FILES: processed_data/complaints_clean.csv.

WEEK 5 DAY 5 TASK

Two classifiers over the complaint corpus: what the complaint is about, and how quickly it must be answered.

MODULE: metermind/message_intelligence/complaint_classifier.py

CONSTANTS: TEXT_MINIMUM_DOCUMENT_COUNT, TEXT_MAXIMUM_FEATURES, HELD_OUT_SHARE, MODEL_SEED, COMPLAINT_MESSAGES_PATH, COMPLAINT_CATEGORY_MODEL_PATH, COMPLAINT_URGENCY_MODEL_PATH, COMPLAINT_SCORES_PATH, COMPLAINT_CATEGORIES (high_bill, no_supply, meter_problem, connection_request, payment_issue), COMPLAINT_URGENCIES (routine, priority, emergency).

FUNCTION: build_text_pipeline()

REQUIREMENTS:

- THE CLEANING IS THE FIRST STEP OF THE PIPELINE, not done beforehand.
- Cleaning step calls clean_messages of Week 5 Day 4.
- Terms weighted (not merely counted), single words and pairs both counted, rare terms dropped, cap on terms kept.
- Decision step settled by the published seed. Returned unfitted.

FUNCTION: split_complaints(corpus, label_column)

STRATIFIED on whichever label is being learned, settled by the published seed. The RAW message text is split, not the cleaned text.

FUNCTION: train_category_classifier(fitting_text, fitting_labels)

Built through build_text_pipeline. BEATS ALWAYS NAMING THE COMMONEST CATEGORY.

FUNCTION: train_urgency_classifier(fitting_text, fitting_labels)

As above, against the urgency label.

FUNCTION: classify_message(category_model, urgency_model, message)

Message handed in raw — both pipelines clean it themselves. Category is one of five published, urgency one of three published.

MAIN BLOCK: Fits both classifiers, saves both, writes their scores, prints them.

> **INPUT:** data/complaint_messages.csv – 5,000 complaints, carrying both labels.

RUN COMMAND:

```
python3 -m metermind.message_intelligence.complaint_classifier
```

CONSOLE OUTPUT:

```
{'category': 0.938, 'urgency': 0.815}
```

GENERATED FILES: artifacts/complaint_category.joblib (read by triage, Week 7 Day 5); artifacts/complaint_urgency.joblib (read by triage and by escalation, Week 6 Day 4); output/complaint_scores.json.

WEEK 6

WEEK 6 DAY 1 TASK

A complaint thread runs to a dozen turns and nobody reads all of them. Today the sentences that carry the thread are found by the attention of Week 5 Day 3, and nothing is invented.

MODULE: metermind/message_intelligence/thread_summary.py

CONSTANTS: SUMMARY_SENTENCE_COUNT, MESSAGE_THREADS_PATH, THREAD_SUMMARIES_PATH.

PUBLISHED NAMES: SENTENCE_END_PATTERN, SHORTEST_SENTENCE_CHARACTERS.

CLASS: UnreadableThread(Exception)

Raised where a thread carries too little to summarise.

FUNCTION: split_sentences(thread_text)

REQUIREMENTS:

- Split at sentence-ending punctuation followed by a space.
- A fragment shorter than the published length is DROPPED — salutations, reference numbers on their own line, one-word acknowledgements are not sentences.
- Every sentence kept is settled to single spaces on one line.
- A thread with nothing long enough returns an empty list.

FUNCTION: sentence_matrix(sentences)

ONE ROW PER SENTENCE. Vocabulary is the thread's own words, lower-cased, trailing punctuation removed. Each row counts word occurrences. Vocabulary in settled order.

FUNCTION: score_sentences(sentences)

A thread of fewer than two sentences raises UnreadableThread. Scores come from attention_weights of Week 5 Day 3, sentences attending over themselves. ONE FINITE SCORE PER SENTENCE.

FUNCTION: summarise_thread(thread_text, sentence_count=None)

REQUIREMENTS:

- NOTHING IN THE SUMMARY IS INVENTED — every sentence returned appears in the thread word for word (extractive).
- A thread with no sentence long enough raises UnreadableThread saying so.
- A thread with no more sentences than wanted comes back whole.
- Sentences chosen are highest scoring, but returned IN THE ORDER THE THREAD SAID THEM.

FUNCTION: write_thread_summaries(threads_path, summaries_path)

Turns of a thread gathered and each cleaned by clean_message of Week 5 Day 4 before joining. A THREAD THAT CANNOT BE SUMMARISED STILL GETS A ROW, carrying why.

MAIN BLOCK: Summarises every thread in the corpus and prints the three counts.

> **INPUT:** data/message_threads.csv – 600 threads across 2,693 turns.

RUN COMMAND:

```
python3 -m metermind.message_intelligence.thread_summary
```

CONSOLE OUTPUT:

```
written: {'threads': 600, 'summarised': 600, 'not summarised': 0}
```

GENERATED FILES: output/thread_summaries.csv.

WEEK 6 DAY 2 TASK

The first real language model. It is asked for a reading no counting can produce, its answers are cached so the same question is never paid for twice, and it is never allowed to fail quietly.

MODULE: metermind/message_intelligence/llm_client.py

Two errors and seven functions today. Week 8 Day 2 adds an eighth, the retry the assistant needs.

CONSTANTS: DEFAULT_LANGUAGE_MODEL, LANGUAGE_MODEL_CACHE_PATH.

CLASS: LanguageModelError(Exception)

The parent of everything that can go wrong with the model.

CLASS: ModelNotConfigured(LanguageModelError)

Raised where no key has been configured. NO NUMBER OF ATTEMPTS SUPPLIES A MISSING KEY (relevant to the retry of Week 8 Day 2).

FUNCTION: load_settings()

The .env file is read into the environment first. A missing or empty key raises ModelNotConfigured naming the variable and where to put it. Model name taken from environment or published default.

FUNCTION: cache_key(model_name, prompt)

Same prompt against same model gives same key every time; ANY DIFFERENCE gives a different key. A SHA-256 digest, not the prompt itself.

FUNCTION: read_cache(cache_path=LANGUAGE_MODEL_CACHE_PATH)

No cache file yet is not an error. A cache file that cannot be read RAISES LanguageModelError rather than being silently treated as empty.

FUNCTION: write_cache(cache, cache_path=LANGUAGE_MODEL_CACHE_PATH)

Parent folder created if not there.

FUNCTION: open_model_client(api_key)

The vendor package is imported INSIDE THIS FUNCTION, not at the top of the module, so a day that never asks the model does not pay to load it.

FUNCTION: ask_model(prompt, cache_path=LANGUAGE_MODEL_CACHE_PATH)

REQUIREMENTS:

- Cache consulted first; a hit returned WITHOUT A CLIENT BEING OPENED AT ALL — must work with no key configured.
- A miss asks the model, adds the answer to the cache before returning.
- THE CLIENT MUST BE HELD FOR AS LONG AS THE CALL TAKES.
- A call that fails raises LanguageModelError. NEVER ANSWERED WITH A DEFAULT, A GUESS OR AN EMPTY STRING.
- A model answering with nothing at all is also an error.

FUNCTION: read_answer(said, wanted_keys)

REQUIREMENTS:

- Handles a fenced JSON block and a preceding sentence — the object is found within what was said.
- An answer carrying no object raises LanguageModelError quoting the beginning of what was said.
- An answer not readable as JSON raises, carrying the reason.
- An answer MISSING ANY WANTED KEY RAISES, NAMING THE MISSING KEYS.

MAIN BLOCK: Prints how many answers the cache is holding.

> **INPUT:** .env must carry GEMINI_API_KEY from this day onwards.

RUN COMMAND:

```
python3 -m metermind.message_intelligence.llm_client
```

CONSOLE OUTPUT:

```
cache holds: 4 answers
```

GENERATED FILES: artifacts/llm_cache.json.

WEEK 6 DAY 3 TASK

A consumer quotes an account number and a meter serial. Today they are found in the text and checked against the register, and the two kinds of fault are told apart.

MODULE: metermind/message_intelligence/reference_extraction.py

CONSTANTS: COMPLAINT_MESSAGES_PATH, CONNECTION_REGISTER_PATH, EXTRACTED_REFERENCES_PATH.

PUBLISHED NAMES: ACCOUNT_NUMBER_PATTERN, METER_SERIAL_PATTERN, COMPLAINT_REFERENCE_PATTERN, EXTRACTED_REFERENCE_FIELDS (message_id, account_numbers, meter_serials, complaint_references, unknown_accounts, serials_on_another_account).

FUNCTION: find_account_numbers(message)

Only whole references match. Each returned once, order settled. A message quoting none returns an empty list. A non-text value is turned into text rather than raising.

FUNCTION: find_meter_serials(message)

As above, against the serial pattern.

FUNCTION: find_complaint_references(message)

As above, against the reference pattern. All three kinds found in one message where one message quotes all three.

FUNCTION: check_against_connection_register(accounts, serials, known_accounts, serial_owner)

REQUIREMENTS:

- An account number the register has never heard of is one fault.
- A SERIAL STANDING AGAINST A DIFFERENT ACCOUNT IS THE WORSE FAULT, reported separately — the first may be a typing mistake, the second is a meter on the wrong connection.
- A serial the register does not carry is neither fault.
- A message quoting a serial but no account cannot have a mismatch.

FUNCTION: extract_and_check(messages_path, register_path, extracted_path)

Register read ONCE. ONE ROW PER MESSAGE, carrying the six published columns. Several references in one column joined so it can be split back apart.

MAIN BLOCK: Extracts and checks the whole corpus and prints what it found.

> **INPUT:** data/complaint_messages.csv and data/connection_register.csv.

RUN COMMAND:

```
python3 -m metermind.message_intelligence.reference_extraction
```

CONSOLE OUTPUT:

```
{'messages': 5000, 'faults': 1116}
```

GENERATED FILES: output/extracted_references.csv.

WEEK 6 DAY 4 TASK

Five thousand messages run in one batch without one bad message stopping it, and the complaints a supervisor must see today are decided on three signals rather than one.

MODULE: metermind/message_intelligence/batch_pipeline.py

CONSTANTS: BATCH_FAILURES_PATH.

PUBLISHED NAME: BATCH_FAILURE_FIELDS – message_id, reason.

CLASS: MessageFailure(Exception)

Constructor __init__(self, message_id, reason): carries which message failed and why, as attributes as well as in the message.

FUNCTION: run_batch(messages, handle_one, failures_path=BATCH_FAILURES_PATH)

REQUIREMENTS:

- ONE RESULT COMES BACK FOR EVERY MESSAGE, in order, whether it succeeded or failed.
- A message that fails is represented by a MessageFailure carrying its own reason, and THE BATCH CARRIES ON.
- Failures written to a file with the two published columns and a header, even where nothing failed.

MAIN BLOCK: Runs two messages, one of which the handler cannot deal with, prints what each came back as.

MODULE: metermind/message_intelligence/escalation.py

CONSTANTS: COMPLAINT_AGE_ESCALATION_DAYS, ESCALATING_URGENCIES, ESCALATING_TONES (emergency, angry), ESCALATION_SIGNALS_NEEDED, TONE_READINGS, TONE_REPLY_KEYS (calm, frustrated, angry), ESCALATION_SAMPLE_MESSAGES, LANGUAGE_MODEL_CACHE_PATH, COMPLAINT_MESSAGES_PATH, COMPLAINT_URGENCY_MODEL_PATH, ESCALATIONS_PATH.

PUBLISHED NAMES: TONE_PROMPT, ESCALATION_FIELDS (message_id, age_days, urgency, tone, escalate, signals).

FUNCTION: complaint_age_days(received_date, today=None)

Day may be supplied, so behaviour can be checked without the answer changing tomorrow.

FUNCTION: decide_escalation(age_days, urgency, tone)

REQUIREMENTS:

- Three signals considered: waited too long, urgency escalates, tone escalates.
- A COMPLAINT ESCALATES ON THE PUBLISHED NUMBER OF SIGNALS, NOT ON ONE.
- The signals that fired returned alongside the decision, named: "waiting too long", "urgency", "tone".
- Moving ANY ONE of the three signals must be able to change the decision.

FUNCTION: read_tone(message_text, cache_path=LANGUAGE_MODEL_CACHE_PATH)

REQUIREMENTS:

- Asked through ask_model (cached).
- Read through read_answer against the published keys.
- A TONE THE PLATFORM DOES NOT PUBLISH RAISES LanguageModelError NAMING IT.
- NO TONE IS GUESSED FROM THE WORDS OF THE COMPLAINT — no keyword list, no fallback.

FUNCTION: escalate_one_message(message, urgency_model, today=None, cache_path=LANGUAGE_MODEL_CACHE_PATH)

Age from complaint_age_days, urgency from the classifier, tone from read_tone, decision from decide_escalation — none repeated here. THE THREE SIGNALS ARE WRITTEN BESIDE THE DECISION.

FUNCTION: write_escalations(messages, urgency_model, escalations_path, today=None, cache_path=LANGUAGE_MODEL_CACHE_PATH)

Run through run_batch, so an unreadable complaint doesn't stop the rest. One row per complaint decided.

MAIN BLOCK: Reads a sample of the corpus, decides each, writes the file, prints the counts.

> **INPUT:** data/complaint_messages.csv, artifacts/complaint_urgency.joblib (Week 5 Day 5), and a key in .env.

RUN COMMAND:

```
python3 -m metermind.message_intelligence.batch_pipeline
```

```
python3 -m metermind.message_intelligence.escalation
```

CONSOLE OUTPUT:

```
handled and returned: 240
MSG000002 failed, and the batch carried on: 'units'
what failed is written to: output/batch_failures.csv
{'decided': 20, 'failed': 0, 'escalated': 3}
```

GENERATED FILES: output/batch_failures.csv; output/escalations.csv.

WEEK 6 DAY 5 TASK

The tariff orders come off the page and are cut into pieces that remember which order and which section they came from. Nothing is searched yet.

MODULE: metermind/retrieval/__init__.py – package marker, empty.

MODULE: metermind/retrieval/document_loader.py

CONSTANTS: TARIFF_ORDERS_DIRECTORY, TARIFF_ORDERS_MANIFEST_PATH.

CLASS: UnreadableOrder(Exception)

Raised where an order cannot be read.

FUNCTION: read_pdf_text(pdf_path)

A file not there raises UnreadableOrder naming the path. A FILE THAT IS THERE BUT YIELDS NO TEXT ALSO RAISES — a scanned order is an order the platform cannot answer from. Every page read and joined in order.

FUNCTION: read_manifest(manifest_path=TARIFF_ORDERS_MANIFEST_PATH)

Returned as it stands.

FUNCTION: load_tariff_orders(orders_directory=TARIFF_ORDERS_DIRECTORY, manifest_path=TARIFF_ORDERS_MANIFEST_PATH)

REQUIREMENTS:

- THE MANIFEST DECIDES WHAT IS LOADED — a document in the folder the manifest does not name is never loaded.
- Each order carries its identifier, title, category, window in force, section headings and text.
- An order named in the manifest whose document cannot be read raises rather than being skipped.

MAIN BLOCK: Loads every order and prints how many were loaded and how much text they hold.

MODULE: metermind/retrieval/chunking.py

CONSTANTS: CHUNK_CHARACTERS, CHUNK_OVERLAP_CHARACTERS, STORED_CHUNK_FIELDS, TARIFF_CHUNKS_PATH.

PUBLISHED NAME: CHUNK_FIELDS – chunk_id, text, and the published metadata fields.

FUNCTION: split_into_chunks(text, size=CHUNK_CHARACTERS, overlap=CHUNK_OVERLAP_CHARACTERS)

REQUIREMENTS:

- No piece longer than the size asked for.
- CONSECUTIVE PIECES OVERLAP BY THE AMOUNT ASKED FOR — a rate and its condition can fall either side of a cut.
- An overlap not smaller than the size raises ValueError (the cut would never advance).
- A piece that is only whitespace is not kept.

FUNCTION: chunk_metadata(order, chunk_text, position)

Identifier carries the order and position. The order, title, category and window in force all travel with the piece. Section is the order's own heading where present, empty otherwise.

FUNCTION: write_chunks(orders, chunks_path=TARIFF_CHUNKS_PATH)

REQUIREMENTS:

- NO TWO PIECES SHARE AN IDENTIFIER, across every order.
- A piece with no heading of its own CARRIES THE HEADING OF THE PIECE BEFORE IT.
- Reports orders cut, pieces produced, and how many pieces ended with no section at all.

MAIN BLOCK: Loads every order, cuts them all, writes the pieces, prints the three counts.

> **INPUT:** data/tariff_orders and data/tariff_orders_manifest.json.

RUN COMMAND:

```
python3 -m metermind.retrieval.document_loader
python3 -m metermind.retrieval.chunking
```

CONSOLE OUTPUT:

```
12 orders, 56819 characters of text
{'orders': 12, 'chunks': 84, 'without_a_section': 0}
```

GENERATED FILES: processed_data/tariff_chunks.json – the 84 pieces with metadata. Read by Week 7 Day 1.

WEEK 7 DAY 1 TASK

The pieces of Week 6 Day 5 go into a vector store and are searched by meaning rather than by the words they happen to use.

MODULE: metermind/retrieval/vector_store.py

Two published names, one error and four functions. Week 7 Day 3 adds the filtered search beside the plain one.

CONSTANTS: EMBEDDING_MODEL_NAME, VECTOR_DISTANCE_MEASURE, TARIFF_STORE_DIRECTORY, TARIFF_STORE_COLLECTION, SEARCH_RESULT_COUNT, STORED_CHUNK_FIELDS, TARIFF_CHUNKS_PATH.

PUBLISHED NAME: STORE_SETTINGS – how the store measures distance, STATED RATHER THAN LEFT TO THE DEFAULT.

PUBLISHED NAME: SEARCH_RESULT_FIELDS – the piece, its text, how far away it was, and the published metadata.

CLASS: TariffStoreMissing(Exception)

Raised where the store has not been built.

FUNCTION: load_embedder(model_name=EMBEDDING_MODEL_NAME)

THE SAME MODEL MUST BE USED TO BUILD THE STORE AND TO SEARCH IT.

FUNCTION: build_tariff_store(chunks_path=TARIFF_CHUNKS_PATH, store_directory=TARIFF_STORE_DIRECTORY, embedder=None)

REQUIREMENTS:

- ONE RECORD IS STORED FOR EVERY PIECE.
- Distance measure declared on the collection from the published settings.
- Each record carries text and published metadata.
- A collection left from an earlier build is REMOVED FIRST.
- Written to disk, not held in memory.

FUNCTION: open_tariff_store(store_directory=TARIFF_STORE_DIRECTORY)

THE STORE IS OPENED, NOT REBUILT — rebuilding on every open would re-embed every piece and make the assistant of Week 8 unusable. No store, or one with no collection of the published name, raises TariffStoreMissing.

FUNCTION: search_tariff(collection, question, embedder, result_count=SEARCH_RESULT_COUNT)

RESULTS ORDERED NEAREST FIRST. Reaches EVERY order in the store, including ones no longer in force — deliberate, so Week 7 Day 3 can show what filtering buys.

MAIN BLOCK: Builds the store, prints how many pieces it holds, searches for the fixed charge on a domestic connection, printing each result's distance, order and section.

> **INPUT:** processed_data/tariff_chunks.json – written by Week 6 Day 5.

RUN COMMAND:

```
python3 -m metermind.retrieval.vector_store
```

The first run downloads the embedding model, about 90 MB, and takes a minute or two. Every run after that is seconds.

CONSOLE OUTPUT:

```
84 chunks stored
  0.538  FY2026-27-LT5     ENERGY CHARGE
  0.540  FY2024-25-LT5     ENERGY CHARGE
  0.545  FY2025-26-LT5     ENERGY CHARGE
  0.555  FY2025-26-LT1     ENERGY CHARGE
  0.556  FY2024-25-LT1     ENERGY CHARGE
```

Five orders reached for one domestic question, three for a category nobody asked about — the problem Week 7 Day 3 solves.

GENERATED FILES: database/tariff_store — the vector store, managed by the store library itself.

WEEK 7 DAY 2 TASK

The model answers from what was retrieved and from nothing else, and every citation it gives must resolve to something it was actually handed.

MODULE: metermind/retrieval/grounded_answer.py

CONSTANTS: GROUNDED_ANSWER_KEYS, UNANSWERABLE_REPLY, LANGUAGE_MODEL_CACHE_PATH, RETRIEVAL_EVALUATION_SET_PATH, TARIFF_ANSWERS_PATH.

PUBLISHED NAMES: CITATION_PATTERN, ANSWER_FIELDS (question_id, question, answer, citations, chunks_handed_over, refused_because), ANSWER_TEMPLATE (instructs the model to use ONLY the extracts, cite every extract relied on and nothing not given, and answer with the published unanswerable reply where extracts don't answer the question).

CLASS: UngroundedAnswer(Exception)

Raised where an answer does not stand on what was retrieved.

FUNCTION: check_citations(answer_text, chunks)

A citation naming a piece not among the extracts raises UngroundedAnswer naming what was invented. AN ANSWER STANDING ON A SOURCE NOBODY SUPPLIED IS WORSE THAN NO ANSWER.

FUNCTION: build_answer_prompt(question, chunks)

NOTHING RETRIEVED MEANS NO ANSWER AT ALL — an empty list raises UngroundedAnswer rather than inviting invention. Every extract handed in appears in the prompt with identifier, order, section and window in force. NO EXTRACT APPEARS THAT WAS NOT HANDED IN.

FUNCTION: answer_from_chunks(question, chunks, cache_path=LANGUAGE_MODEL_CACHE_PATH)

Prompt built, model asked through ask_model, answer read through read_answer. THE CITATIONS ARE CHECKED TWICE: those in the answer's text, and those in the citation field.

FUNCTION: write_answers(questions, collection, embedder, answers_path, cache_path=LANGUAGE_MODEL_CACHE_PATH)

A QUESTION THAT COULD NOT BE ANSWERED STILL GETS A ROW, carrying why. Each row carries what the answer stands on.

MAIN BLOCK: Answers the first five questions of the evaluation set and prints the counts.

> **INPUT:** data/retrieval_evaluation_set.json, the store from Week 7 Day 1, and a key in .env.

RUN COMMAND:

```
python3 -m metermind.retrieval.grounded_answer
```

CONSOLE OUTPUT:

```
{'questions': 5, 'answered': 5, 'refused': 0}
```

With no key the search still runs and only the answering refuses.

GENERATED FILES: output/tariff_answers.csv.

WEEK 7 DAY 3 TASK

The same question has different answers in different billing periods. Today retrieval is narrowed to the order that was actually in force, and asking about a period no order covers is refused rather than answered from a dead order.

MODULE: metermind/retrieval/vector_store.py (continued)

FUNCTION: search_with_filter(collection, question, embedder, conditions, result_count=SEARCH_RESULT_COUNT)

THE FILTER IS APPLIED BY THE STORE, NOT AFTER THE SEARCH — retrieving the five nearest and discarding non-matches leaves fewer than five, sometimes none. Conditions of nothing search everything. Results carry the published fields, ordered nearest first.

FUNCTION: search_tariff – edited today, now calls search_with_filter with no conditions rather than querying the store itself.

MODULE: metermind/retrieval/filtered_search.py

CONSTANTS: SEARCH_RESULT_COUNT, LANGUAGE_MODEL_CACHE_PATH, RETRIEVAL_EVALUATION_SET_PATH, FILTERED_ANSWERS_PATH.

PUBLISHED NAME: FILTERED_ANSWER_FIELDS – question_id, question, consumer_category, billing_period, order_in_force, answer, citations, refused_because.

CLASS: NoOrderInForce(Exception)

Raised where no order covered the period asked about.

FUNCTION: order_in_force(manifest, consumer_category, billing_period)

REQUIREMENTS:

- Returns the order whose window HOLDS the period, for that category — windows tile without gap or overlap, so exactly one qualifies.
- The same question in two different periods reaches two different orders.
- A PERIOD NO ORDER COVERS RAISES NoOrderInForce — answering from an order that had ceased to have effect would be worse than not answering.

FUNCTION: filtered_search(collection, question, embedder, consumer_category, billing_period, manifest=None, result_count=SEARCH_RESULT_COUNT)

Order settled by order_in_force first, search narrowed to it. NO PIECE COMES BACK FROM AN ORDER THAT WAS NOT IN FORCE, and none from another category. A period no order covers raises rather than searching everything.

FUNCTION: write_filtered_answers(questions, collection, embedder, answers_path, manifest=None, cache_path=LANGUAGE_MODEL_CACHE_PATH)

Each question answered from its own order in force. THE ORDER USED IS WRITTEN INTO THE ROW. A question that could not be answered still gets a row carrying why.

MAIN BLOCK: Shows the same question searched three ways — unfiltered, narrowed to the domestic order in force in a named period, and asked about a period no order covers — then answers five questions with the filter on.

> **INPUT:** data/retrieval_evaluation_set.json, data/tariff_orders_manifest.json, the store from Week 7 Day 1, and a key in .env.

RUN COMMAND:

```
python3 -m metermind.retrieval.filtered_search
```

CONSOLE OUTPUT:

```
nearest, any order at all : ['FY2024-25-LT5', 'FY2025-26-AM-LT1', 'FY2025-26-AM-LT5', 'FY2025-26-LT5', 'FY2026-27-LT5']
nearest, domestic 2024-09 : ['FY2024-25-LT1']
asked about 2019-01       : no domestic order was in force in 2019-01; answering from an order that had ceased to have effect would
be worse than not answering
five questions, filtered  : {'questions': 5, 'answered': 5, 'refused': 0}
```

One question, three answers: unfiltered reaches five orders, four for a category nobody asked about; filtered reaches exactly one; a period no order covers is refused. With no key configured the last line instead reads {'questions': 5, 'answered': 0, 'refused': 5}.

GENERATED FILES: output/filtered_answers.csv.

WEEK 7 DAY 4 TASK

Four earlier days are wrapped as tools an assistant may call. Each declares what it takes, refuses a malformed argument before its body runs, and says what it is for.

MODULE: metermind/core/exceptions.py (continued)

CLASS: ToolError(MeterMindError)

Raised where a tool cannot answer.

MODULE: metermind/assistant/__init__.py – package marker, empty.

MODULE: metermind/assistant/tools.py

CONSTANTS: CONSUMER_CATEGORIES, BILLING_PERIOD_PATTERN, CURRENCY_DECIMAL_PLACES, READINGS_DATABASE_PATH, CHAMPION_MODEL_PATH, OPERATING_RULE_PATH, EXCEPTION_REPORT_PATH, TARIFF_STORE_DIRECTORY (each held module-level so a testcase can point tools at its own).

PUBLISHED NAMES: READING_TOOLS, opened_resources – READING_TOOLS is the four tools, the only list the assistant of Week 8 may call from. opened_resources is empty until first call, lazily loaded.

CLASS: ReadingRequest(BaseModel)

Field: reading_identifier, carrying a description. EVERY FIELD CARRIES A DESCRIPTION — the planner of Week 8 Day 1 is shown these names and descriptions.

CLASS: BillRequest(BaseModel)

Fields: consumer_category (one of published categories), units_consumed (not below zero), sanctioned_load_kw (above zero). Bounds declared on fields.

VALIDATOR a_category_that_is_billed: A category outside the published three raises, NAMING THE THREE.

CLASS: TariffRequest(BaseModel)

Fields: question, consumer_category, billing_period (matching published shape).

VALIDATOR a_category_with_a_schedule: as above.

FUNCTION: look_up_reading(reading_identifier)

Fetched from the database of Week 2 Day 2. AN IDENTIFIER NOTHING IS STORED AGAINST RAISES ToolError NAMING IT.

FUNCTION: calculate_bill(consumer_category, units_consumed, sanctioned_load_kw)

THE ANSWER IS WEEK 1'S — calls calculate_bill_amount, does no arithmetic of its own.

FUNCTION: predict_reading_exception(reading_identifier)

Reading fetched through look_up_reading; widened as Week 4 Day 3; classified under the saved operating rule. THE PRECISION IS READ OUT OF THE REPORT WRITTEN BY WEEK 5 DAY 2, not stated in this module.

FUNCTION: search_tariff_order(question, consumer_category, billing_period)

Goes through filtered_search of Week 7 Day 3, so ONLY THE ORDER IN FORCE IS REACHED. A period no order covers raises ToolError carrying the reason.

MAIN BLOCK: Prints each tool with the argument names it takes.

RUN COMMAND:

```
python3 -m metermind.assistant.tools
```

CONSOLE OUTPUT:

```
look_up_reading            ['reading_identifier']
calculate_bill             ['consumer_category', 'sanctioned_load_kw', 'units_consumed']
predict_reading_exception  ['reading_identifier']
search_tariff_order        ['billing_period', 'consumer_category', 'question']
```

GENERATED FILES: None.

WEEK 7 DAY 5 TASK

What travels between the nodes of a graph, and the first two nodes: the triage that reads the question and the runner that calls what a plan named.

MODULE: metermind/assistant/state.py

PUBLISHED NAME: ACCUMULATING_CHANNELS – the two channels that ADD rather than replace.

CLASS: AssistantState(TypedDict)

Fields: question, consumer_category, billing_period (text); category, urgency (what triage read); plan (what the planner asked for); answer, refusal (how the run ended); finished (boolean); steps, tool_calls (the two accumulating channels — each node appends one word to steps, the name it is registered under in the graph of Week 8 Day 2: triage, plan, tools, draft, critique, gate, respond).

REQUIREMENTS:

- EVERY CHANNEL IS DECLARED, whether or not a given run uses it — a node returning an undeclared channel is silently discarded.
- EXACTLY TWO CHANNELS ACCUMULATE: steps and tool_calls. Every other channel is replaced by whichever node last wrote it.
- The state is widened twice in Week 8; nothing declared today is removed.

FUNCTION: new_assistant_state(question, consumer_category="domestic", billing_period="")

THE QUESTION IS THE ONE THING A NEW RUN IS GIVEN — every other channel starts empty, both accumulating channels start as empty lists, run starts unfinished.

MAIN BLOCK: Prints every channel the state declares, and a new state.

MODULE: metermind/assistant/nodes.py

Three published names and one node today. The tool runner arrives on Week 8 Day 1 and the approval gate on Week 8 Day 2.

CONSTANTS: COMPLAINT_CATEGORY_MODEL_PATH, COMPLAINT_URGENCY_MODEL_PATH (module-level).

PUBLISHED NAME: loaded_classifiers – empty until the first question.

FUNCTION: triage_the_question(state)

REQUIREMENTS:

- A QUESTION THAT IS EMPTY OR ONLY WHITESPACE IS REFUSED, WITH A REASON, AND THE RUN IS FINISHED. It does not raise; the graph must route a refusal to the end like any other outcome.
- Category and urgency come from the classifiers of Week 5 Day 5 — no keyword list, no fallback reading.
- Classifiers not on file raise ToolError naming where they were looked for.
- Loaded once and kept.
- The node returns ONLY THE CHANNELS IT CHANGED, and records that it ran.

MAIN BLOCK: Triages three questions — ordinary, urgent, empty — and prints each.

> **INPUT:** artifacts/complaint_category.joblib and artifacts/complaint_urgency.joblib from Week 5 Day 5.

RUN COMMAND:

```
python3 -m metermind.assistant.state
python3 -m metermind.assistant.nodes
```

CONSOLE OUTPUT:

```
['answer', 'billing_period', 'category', 'consumer_category', 'finished', 'plan', 'question', 'refusal', 'steps', 'tool_calls',
'urgency']
{'question': 'Why is my bill so high this month?', 'consumer_category': 'domestic', 'billing_period': '', 'category': '', 'urgency':
'', 'plan': '', 'answer': '', 'refusal': '', 'finished': False, 'steps': [], 'tool_calls': []}
'Why is my bill for June three times the usual amount'   -> {'category': 'high_bill', 'urgency': 'routine', 'steps': ['triage']}
'There has been no supply in our street since last ev'   -> {'category': 'no_supply', 'urgency': 'priority', 'steps': ['triage']}
''                                                       -> {'refusal': 'nothing was asked', 'finished': True, 'steps': ['triage']}
```

GENERATED FILES: None.

WEEK 8 DAY 1 TASK

Two agents and the ordinary code between them: one that chooses which tools to call and says why, one that drafts the reply from what came back, and a plain node that dispatches what was chosen.

MODULE: metermind/assistant/agents.py

Seven published names and three functions. The critic and responder arrive on Week 8 Day 2.

CONSTANTS: LANGUAGE_MODEL_CACHE_PATH (module-level), ASSISTANT_TOOL_ROUNDS.

PUBLISHED NAMES: PLACEHOLDER_PATTERN, PLANNER_KEYS, RESOLVER_KEYS, TOOL_CATALOGUE (built from READING_TOOLS of Week 7 Day 4, keyed by tool name, each entry carrying "does" and "arguments"), PLANNER_PROMPT, RESOLVER_PROMPT.

TOOL_CATALOGUE is not written by hand — it is read off the tools themselves, so a tool renamed, described differently or given a different argument is offered to the planner exactly as it now stands. A catalogue naming a tool's arguments as anything other than what its schema declares produces plans that fail validation on every call, silently.

Each prompt is a template with named placeholders, published as a name, not built inline, so a testcase can read what the agents are told without calling a model. PLANNER_PROMPT carries placeholders: catalogue, rounds, question, category, urgency, consumer_category, billing_period. It instructs the model to reply with JSON holding plan (list of at most "rounds" steps, each with tool and arguments) and reason; forbids naming any tool outside the catalogue; requires exact argument names; directs leaving a step out rather than guessing; allows an empty plan. RESOLVER_PROMPT carries: question, tool_calls. It confines the draft to what the tools returned, forbids working out any figure the tools did not return, requires saying plainly where the tools did not answer, and instructs a JSON reply holding draft and reason.

FUNCTION: build_agent_prompt(template, filling)

A TEMPLATE WITH A PLACEHOLDER NOTHING WAS SUPPLIED FOR RAISES LanguageModelError NAMING WHAT IS MISSING — it does not fill what it can and send the rest. Every one of the four agents builds its prompt through this one function.

FUNCTION: plan_tools(state, cache_path=None)

REQUIREMENTS:

- THE PLAN COMES FROM THE MODEL AND FROM NOWHERE ELSE — no keyword rule, no default plan, no fallback.
- Model offered TOOL_CATALOGUE, the round ceiling, the question, what triage read, and the connection asked about.
- Reply read through the published reader against PLANNER_KEYS.
- A PLAN THAT IS NOT A LIST OF STEPS RAISES LanguageModelError SAYING WHAT CAME BACK INSTEAD.
- Returns only the channels it changed, records that it ran.

FUNCTION: draft_answer(state, cache_path=None)

THE DRAFT RESTS ON THE TOOL RESULTS AND ON NOTHING ELSE. Reply read through the published reader against RESOLVER_KEYS.

MAIN BLOCK: Prints what each agent answers with and is told, the tools the planner may name with their argument names, and a note that the resolver sees only what the tools returned.

MODULE: metermind/assistant/nodes.py (continued)

FUNCTION: run_tools(state)

REQUIREMENTS:

- A PLAN NAMING A TOOL THAT DOES NOT EXIST IS REFUSED BEFORE ANYTHING RUNS — not the tools up to the bad one, nothing at all.
- Tools that do exist are called in the order the plan named.
- A TOOL THAT FAILS IS RECORDED WITH ITS ERROR AND THE RUN CARRIES ON.
- Every call recorded with tool, arguments and result or error.
- Only tools in the published list may be called.

MODULE: metermind/assistant/state.py (continued)

Edited today — no new top-level names. plan is declared as a list, not text. draft is added, as text. Everything from Week 7 Day 5 stays declared.

> **INPUT:** artifacts/complaint_category.joblib and artifacts/complaint_urgency.joblib from Week 5 Day 5.

RUN COMMAND:

```
python3 -m metermind.assistant.agents
python3 -m metermind.assistant.state
```

CONSOLE OUTPUT:

```
planner    answers with ('plan', 'reason')       and is told ['catalogue', 'rounds', 'question', 'category', 'urgency',
'consumer_category', 'billing_period']
resolver   answers with ('draft', 'reason')      and is told ['question', 'tool_calls']

the planner may name these 4 tools and no others, and is given the argument names each one takes:
   look_up_reading            Find one reading on file by its identifier: the meter, the account,
                              the dates, the units and everything recorded against the cycle.
                              takes ['reading_identifier']
   calculate_bill             Work out what a billing cycle costs under the schedule in force:
                              the slab-by-slab energy charge, the fixed charge on the sanctioned
                              load, the electricity duty and the rebate.
                              takes ['consumer_category', 'sanctioned_load_kw', 'units_consumed']
   predict_reading_exception  Say which of the five exception classes a reading on file belongs
                              to, and how often that answer is right.
                              takes ['reading_identifier']
   search_tariff_order        Find what the tariff order that was actually in force for that
                              consumer category in that billing period says.
                              takes ['billing_period', 'consumer_category', 'question']

the resolver is handed what the tools returned, and nothing else

['answer', 'billing_period', 'category', 'consumer_category', 'draft', 'finished', 'plan', 'question', 'refusal', 'steps',
'tool_calls', 'urgency']
{'question': 'Why is my bill so high this month?', 'consumer_category': 'domestic', 'billing_period': '', 'category': '', 'urgency':
'', 'plan': [], 'draft': '', 'answer': '', 'refusal': '', 'finished': False, 'steps': [], 'tool_calls': []}
```

GENERATED FILES: None.

WEEK 8 DAY 2 TASK

The last two agents, the gate a run stops at, the graph that wires all seven nodes together, and the retry the four agents ask through.

MODULE: metermind/message_intelligence/llm_client.py (continued)

CONSTANTS: MODEL_RETRY_ATTEMPTS, MODEL_RETRY_SECONDS.

FUNCTION: ask_model_with_retry(prompt, cache_path=LANGUAGE_MODEL_CACHE_PATH, attempts=MODEL_RETRY_ATTEMPTS, pause_seconds=MODEL_RETRY_SECONDS, waits=None)

REQUIREMENTS:

- EACH WAIT IS LONGER THAN THE ONE BEFORE IT.
- ModelNotConfigured IS RE-RAISED AT ONCE AND NEVER RETRIED — trying three times for a key that is not there wastes time and tells nothing new.
- After the last attempt the failure is raised rather than swallowed.
- Where a list is supplied for the waits, they are recorded in it and not taken.
- Goes through the published asker of Week 6 Day 2, so the cache is still read and written exactly as before.

MODULE: metermind/assistant/agents.py (continued)

PUBLISHED NAMES: CRITIC_KEYS, RESPONDER_KEYS, CRITIC_PROMPT, RESPONDER_PROMPT.

CRITIC_PROMPT carries placeholders: question, tool_calls, draft. States the critic may only object — may not rewrite the draft, may not answer the question itself. Names what is worth objecting to: a figure the tools did not return, a statement contradicting what they did return, an accusation against the consumer the tools did not establish. Instructs JSON reply holding veto (true/false) and reason. RESPONDER_PROMPT carries: question, draft. Confines the reply to what the draft says, in plain language, no jargon, no figure the draft doesn't carry. Instructs JSON reply holding reply and reason.

FUNCTION: critique_draft(state, cache_path=None)

REQUIREMENTS:

- THE CRITIC WRITES NO ANSWER AND NO DRAFT — it returns an objection or nothing to object to.
- Reply read through the published reader against CRITIC_KEYS.
- AN OBJECTION IS CARRIED AS THE REASON THE MODEL GAVE, NOT AS A FLAG.
- Where the model does not object, the objection channel is left empty.

FUNCTION: write_response(state, cache_path=None)

Responder handed the question and the draft, nothing else — not the tool results, since it may not add anything the draft doesn't carry. Reply read against RESPONDER_KEYS. THIS NODE FINISHES THE RUN.

plan_tools and draft_answer edited to ask through ask_model_with_retry rather than the plain asker; nothing else about either changes.

MAIN BLOCK: Prints what each of the four agents answers with and is told, the tools with argument names, and the responder's prompt filled with a question and a draft.

MODULE: metermind/assistant/nodes.py (continued)

CONSTANTS: HELD_EXCEPTION_CLASSES, HELD_FOR_A_PERSON_REPLY.

FUNCTION: approval_gate(state)

REQUIREMENTS:

- THE GATE NEVER READS THE DRAFT'S WORDING — it holds on what the run established, not on how the reply is phrased.
- A RUN WHOSE TOOLS READ THE CONNECTION AS ONE OF THE HELD CLASSES IS HELD — those two of the five exception classes carry a legal consequence for the consumer. The other three do not hold the run.
- A DRAFT THE CRITIC OBJECTED TO IS HELD, CARRYING THE OBJECTION.
- A RUN THAT DRAFTED NOTHING – empty or only whitespace – IS HELD.
- Where more than one tripped, the hold names all of them.
- A HELD RUN STILL TELLS THE CONSUMER SOMETHING — the published reply goes back as the answer, the hold carries the reason for the inspector.
- An ordinary run is held for nothing and passes straight through.

MODULE: metermind/assistant/state.py (continued)

Edited today — no new top-level names. veto (text, the critic's objection or empty) and held (text, why a run is waiting, or empty) added.

MODULE: metermind/assistant/graph.py

Two published names and three functions. This is the module that wires the seven nodes into one flow.

PUBLISHED NAMES: CARRY_ON, ASSISTANT_NODES – ASSISTANT_NODES is the seven pairs (name, function) in run order: triage, plan, tools, draft, critique, gate, respond.

FUNCTION: worth_answering(state)

Returns the end where the run is finished or carries a refusal, and CARRY_ON otherwise. IT DECIDES ON THE STATE AND NOT ON THE QUESTION — no re-reading, no calling anything.

FUNCTION: still_going(state)

As above, wired at three different branches — a separate name from the router after triage, since the two ask the same question at different points and a change to one must not silently change the other.

FUNCTION: build_assistant_graph()

REQUIREMENTS:

- Built over the published state.
- EVERY NODE IN ASSISTANT_NODES IS REGISTERED AND NOTHING ELSE IS.
- Flow enters at the triage.
- THE FOUR BRANCHES ARE CONDITIONAL AND THE THREE OTHERS ARE NOT: after triage, after tools, after critique and after gate the run may leave for the end; plan→tools, draft→critique, respond→end may not.
- Returned uncompiled.

MAIN BLOCK: Prints every edge of the compiled flow, marking the conditional ones, and what each router returns for an ordinary, a stopped and a refused run.

RUN COMMAND:

```
python3 -m metermind.assistant.graph
```

CONSOLE OUTPUT:

```
the flow, and where it can leave early:
   __start__     -> triage
   critique      -> __end__
   critique      -> gate          only when the run is still going
   draft         -> critique
   gate          -> __end__
   gate          -> respond       only when the run is still going
   plan          -> tools
   tools         -> __end__
   tools         -> draft         only when the run is still going
   triage        -> __end__
   triage        -> plan          only when the run is still going
   respond       -> __end__
   an ordinary run          worth_answering: carry on   still_going: carry on
   a run that has stopped   worth_answering: __end__    still_going: __end__
   a run that was refused   worth_answering: __end__    still_going: __end__
```

GENERATED FILES: None.

WEEK 8 DAY 3 TASK

The graph compiled against a saver so a held run can be picked up where it stopped, and the whole assistant behind one authenticated door.

MODULE: metermind/assistant/workflow.py

CONSTANTS: ASSISTANT_THREADS_PATH.

PUBLISHED NAME: ASSISTANT_OUTCOME_FIELDS – thread_id, answer, held, refusal, category, urgency, steps, tools_called.

CLASS: NoSuchThread(MeterMindError)

Raised where a thread is asked about that nobody started, or that is not waiting for anybody.

FUNCTION: open_saved_graph(threads_path=ASSISTANT_THREADS_PATH)

REQUIREMENTS:

- THE RUN IS WRITTEN DOWN, NOT HELD IN MEMORY — a run held in a running process is lost when the process restarts.
- Directory created if not there.
- Connection opened so it may be used from more than one thread, since the service serves more than one request at a time.
- THE COMPILED GRAPH STOPS BEFORE THE NODE THAT WRITES THE CONSUMER'S REPLY — the last point at which a held run can still be caught.

FUNCTION: run_assistant(question, thread_id, consumer_category="domestic", billing_period="", graph=None)

REQUIREMENTS:

- Run identified by its thread, so two consumers asking at once do not read each other's answers.
- A question starts a new run; nothing given in its place carries on the run already on that thread.
- A RUN THAT STOPPED AT THE GATE AND WAS NOT HELD IS CARRIED ON TO THE END WITHOUT ANYBODY BEING ASKED. Stopping before the last node is how a hold is caught; it is not, by itself, a hold.
- THE OUTCOME CARRIES THE CONSUMER'S ANSWER AND THE INSPECTOR'S REASON SIDE BY SIDE, and exactly the published fields.
- Tools called reported by name, not the whole record.
- Any channel the run never set comes back empty rather than missing.

FUNCTION: release_held_run(thread_id, graph=None)

REQUIREMENTS:

- A THREAD NOBODY STARTED RAISES NoSuchThread, and so does a thread whose run already finished and is waiting for nobody.
- The hold is cleared and the run carried on from where it stopped — not restarted from the question, since everything the run established is already on the thread.
- Released run comes back in the same shape as any other.

MAIN BLOCK: Prints where threads are kept, the node a run stops before, what a run comes back as, and what an unused thread holds.

MODULE: metermind/api/routers/assistant.py

Two published names, two schemas and two routes. No main block: reached through the service.

CONSTANTS: ASSISTANT_THREADS_PATH (module-level).

PUBLISHED NAMES: router, opened_graph – opened_graph is empty until opened, so the graph is opened once for the life of the service.

CLASS: AssistantQuestion(BaseModel)

AN EMPTY QUESTION IS REFUSED BY THE SCHEMA, before a thread is started and before any model is asked. A question sent with no thread is a new run; sent with one carries on that thread.

CLASS: AssistantReply(BaseModel)

IT CARRIES EXACTLY THE PUBLISHED OUTCOME FIELDS. The consumer reads the answer; the inspector reads the rest.

ROUTE: POST /assistant/questions

ask_assistant(asked: AssistantQuestion, carried=Depends(require_signed_in))

REQUIREMENTS:

- ANY SIGNED-IN ROLE MAY ASK — it is the consumer's own question.
- Where no thread is sent, a new one is made, and comes back in the reply.
- Graph opened once and kept.
- THE REPLY IS THE OUTCOME OF THE RUN, UNCHANGED — this route decides nothing about what the consumer is told, that was decided at the gate.
- A MODEL THAT CANNOT BE REACHED ANSWERS 503, CARRYING WHY — not 500, since it is the platform being temporarily unable to answer, not the caller having asked something wrong.

RETURNS: 200 with an AssistantReply. 401 without a token, 422 where the question is empty, 503 where the model cannot be reached.

ROUTE: POST /assistant/threads/{thread_id}/release

release_assistant_thread(thread_id: str, carried=Depends(require_inspector))

REQUIREMENTS:

- ONLY AN INSPECTOR MAY RELEASE A HELD RUN — a consumer releasing their own hold would make the gate ceremonial.
- A THREAD THAT IS NOT WAITING ANSWERS 404, CARRYING WHY — not 500.
- Released, the run finishes and the reply goes out in the same shape as any other.
- A MODEL THAT CANNOT BE REACHED ANSWERS 503 HERE TOO, and the run stays on its thread to be released again once it can be.

RETURNS: 200 with an AssistantReply. 401 without a token, 403 for a role that is not an inspector, 404 for a thread not waiting, 503 where the model cannot be reached.

MODULE: metermind/api/application_factory.py – edited today, no new top-level names. The assistant router is mounted under API_PREFIX, and the service is complete.

> **INPUT:** Everything the four tools of Week 7 Day 4 reach: database/metermind.sqlite (Week 2 Day 2), artifacts/champion_exception.joblib and output/operating_rule.json (Week 4 Day 4), output/exception_report.csv (Week 5 Day 2), the vector store under database/tariff_store (Week 7 Day 1). Also artifacts/complaint_category.joblib and artifacts/complaint_urgency.joblib for triage. Answering a question needs GEMINI_API_KEY set in .env.

RUN COMMAND:

```
python3 -m metermind.assistant.workflow
```

Then, with the service running and a consumer's token in hand:

```
curl -s -X POST http://127.0.0.1:8000/api/v1/assistant/questions \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"question":"Why is reading RDG20000011 billed at that amount?",
       "consumer_category":"domestic","billing_period":"2025-05","thread_id":"thread-one"}'
```

CONSOLE OUTPUT:

```
threads are kept in : database/assistant_threads.sqlite
a run stops before  : respond, so a held one can be picked up there
a run comes back as : ('thread_id', 'answer', 'held', 'refusal', 'category', 'urgency', 'steps', 'tools_called')
an unused thread has: nothing on it and waits at ()
```

A question naming a reading the champion reads as one of the held classes comes back with answer set to the published held reply, held naming the class that stopped it, and steps ending at gate with no respond. An inspector releasing that thread gets the same reply shape back with held cleared and the answer written. A question sent with no token answers 401, and releasing a thread nobody started answers 404.

GENERATED FILES: database/assistant_threads.sqlite — every run, by thread, so a held one can be picked up where it stopped.

CLOSING SECTIONS

OUTPUT

Thirty-seven of the thirty-eight days print something; Week 1 Day 3 is the one that does not, because none of its three modules carries a demonstration. Fifteen of them write no file at all — they deliver library code a later day calls, or a route that answers over the network, and they are validated by their own testcases and by what the main block prints.

Nothing under data or config is ever written to. Every generated file lands in processed_data, database, artifacts or output, all four of which are supplied empty.

Running tests.py also writes into output and database. Testcases that start the real application install the request log of Week 3 Day 2, and the ones that move a case append to the audit trail of Week 3 Day 3. Finding test traffic in output/logs/requests.log and output/audit_trail.csv beside your own runs is expected and harmless.

FLOW DIAGRAM

Original given as two screenshot images. Reconstructed below as the exact edge list from Week 8 Day 2's build_assistant_graph (this is the literal console output the spec itself gives for that day, rendered here as a diagram-style flow instead).

```
START
  |
  v
[triage] --(refused / empty question)--> END
  |  (carry on)
  v
[plan]
  |
  v
[tools] --(refused: unknown tool named)--> END
  |  (carry on)
  v
[draft]
  |
  v
[critique] --(finished/refused upstream)--> END
  |  (carry on)
  v
[gate] --(finished/refused upstream)--> END
  |  (carry on)
  v
[respond] --> END

Held at [gate]: exception class in HELD_EXCEPTION_CLASSES, OR critic vetoed,
                OR draft was empty. Consumer still gets HELD_FOR_A_PERSON_REPLY;
                inspector releases via POST /assistant/threads/{id}/release,
                which resumes from gate straight to respond.
```

FILES PRODUCED — WEEK 1 TO WEEK 8

Original given as screenshot table images (Week 1–4 shown explicitly; the spec text describes the same for Week 5–8 in each day's own GENERATED FILES section, transcribed faithfully above). Consolidated here from every day's own GENERATED FILES line already given in this document.

Week	Files generated
1	none (pure library code)
2	processed_data/readings_clean.csv, processed_data/readings_rejected.csv, database/metermind.sqlite, output/reading_reports.csv, database/metermind_orm.sqlite
3	output/logs/requests.log, output/audit_trail.csv, artifacts/baseline_exception.joblib, output/baseline_scores.json
4	artifacts/tree_exception.joblib, artifacts/forest_exception.joblib, output/cross_validation.csv, processed_data/readings_widened.csv, output/feature_lift.json, artifacts/champion_exception.joblib, output/operating_rule.json, output/queue_no_action.csv, output/queue_meter_reading.csv, output/queue_metering.csv, output/queue_vigilance.csv, output/queue_billing.csv
5	output/exception_report.csv, processed_data/complaints_clean.csv, artifacts/complaint_category.joblib, artifacts/complaint_urgency.joblib, output/complaint_scores.json
6	output/thread_summaries.csv, artifacts/llm_cache.json, output/extracted_references.csv, output/batch_failures.csv, output/escalations.csv, processed_data/tariff_chunks.json
7	database/tariff_store, output/tariff_answers.csv, output/filtered_answers.csv
8	database/assistant_threads.sqlite

EXPECTED OUTPUT

The thirty-eight days build one service. This is that service running: started, an account created, a token obtained, a reading submitted that cannot have happened, a reading submitted that can, that reading read back and listed, one that is not on file refused, a case moved along the ladder and then refused going backwards, a screening called without a token and then with one, and the assistant asked a question.

Every one of the fourteen calls goes through the middleware of Week 3 Day 2. The four routes written on Week 2 carry no token — they were written twelve days before a token existed — and every route added from Week 3 Day 4 on is declared with the guard it needs.

STARTING THE SERVICE:

```
python3 -m metermind.api.application_factory
INFO:     Started server process [28874]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

GET /api/v1/health – HTTP 200. The one route that needs nothing at all.

```
{"service":"metermind","status":"up"}
```

POST /api/v1/accounts – HTTP 201. Password never echoed back.

```
{"account_number":"ACC10000123","email_address":"meena@example.in","role":"inspector"}
```

POST /api/v1/accounts/login – HTTP 200.

```
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWI...","token_type":"bearer",
 "role":"inspector"}
```

POST /api/v1/readings with the current reading below the previous – HTTP 201. Accepted, priced and stored; not billable, reason carried rather than raised.

```
{"reading_id":"RDG50000001","account_number":"ACC10000123","consumer_category":"domestic",
 "units_consumed":0.0,"energy_charge_inr":0.0,"fixed_charge_inr":150.0,"net_bill_inr":150.0,
 "is_billable":false,"refusal_reasons":["the current reading is below the previous reading and
 the meter cannot have rolled over"]}
```

POST /api/v1/readings with the same cycle the right way round – HTTP 201.

```
{"reading_id":"RDG50000002","account_number":"ACC10000123","consumer_category":"domestic",
 "units_consumed":467.0,"energy_charge_inr":2575.7,"fixed_charge_inr":150.0,
 "net_bill_inr":2854.48,"is_billable":true,"refusal_reasons":[]}
```

Both replies carry the fixed charge of Rs. 150.00 on a 3 kW connection, since that charge is on the sanctioned load and not on what was used.

GET /api/v1/readings/RDG50000002 – HTTP 200. The same reply the submission returned.

GET /api/v1/readings?page=1&page_size=2 – HTTP 200.

```
{"readings":[{"reading_id":"RDG50000001", ...},{"reading_id":"RDG50000002", ...}],
 "page":1,"page_size":2,"total":2}
```

GET /api/v1/readings/RDG99999999 – HTTP 404.

```
{"detail":"there is no reading RDG99999999"}
```

PATCH /api/v1/readings/RDG50000002/status to assigned – HTTP 200, inspector only.

```
{"reading_id":"RDG50000002","from_status":"raised","to_status":"assigned",
 "recorded_by":"je/1180"}
```

PATCH /api/v1/readings/RDG50000002/status back to raised – HTTP 409. The ladder only goes one way.

```
{"detail":"a case at assigned may move to inspected, closed_no_fault, not to raised"}
```

POST /api/v1/screening with no token – HTTP 401.

```
{"detail":"the token could not be read"}
```

POST /api/v1/screening with a token – HTTP 200. First call takes ~2/3 second (champion read off disk); every screening after is answered from the model already in memory.

```
{"account_number":"ACC10000123","billing_period":"2026-06",
 "exception_class":"genuine_high_usage","confidence":0.6514,"goes_to":"queue_no_action.csv"}
```

POST /api/v1/assistant/questions with no key in .env – HTTP 503. Nothing is broken; the model is not reachable.

```
{"detail":"the assistant could not answer: GEMINI_API_KEY is not set; put it in .env before
 asking the model anything"}
```

POST /api/v1/assistant/questions with a key in .env – HTTP 200. Request:

```
{"question": "Why is my June bill so high?",
 "consumer_category": "domestic", "billing_period": "2026-06"}
```

All seven nodes ran. Triage read the question as high_bill at routine urgency; the planner chose three tools; two refused what they were handed and the third answered; the critic passed the draft; the gate found nothing to hold it on.

```
"category": "high_bill", "urgency": "routine",
"steps": ["triage","plan","tools","draft","critique","gate","respond"],
"tools_called": ["look_up_reading","calculate_bill","search_tariff_order"],
"held": "", "refusal": ""
```

What the consumer is sent, in answer:

```
We cannot explain your high June bill because we do not have access to your specific meter
reading or billing records. Your bill is made up of a fixed charge of Rs. 50.00 per kilowatt of
your approved load, energy charges with a 5% electricity tax, and a discount of Rs. 1.50 per unit
if you use 100 units or less in a billing cycle. If you think there is a mistake, you can contact
our Consumer Grievance Redressal Forum.
```

That run is worth reading twice, because it is the platform behaving correctly under a question it cannot fully answer. The question named no reading, so the planner had no identifier to hand look_up_reading and invented one, which came back "there is no reading RDG202606 on file"; calculate_bill was handed a sanctioned load of zero and was refused by its own schema; only search_tariff_order answered, with the order in force for a domestic connection in that period. The resolver was handed all three outcomes, including the two failures, and said plainly that the readings were not available rather than producing a figure.

A question that names a reading on file behaves differently. RDG20000011 is a domestic cycle of 560.9 units that the champion reads as genuine_high_usage: the tools answer, the draft carries real figures, and the gate lets it through. RDG20000034 is a domestic cycle of 76.7 units that the champion reads as tamper_suspected: the run comes back with answer set to the published held reply, held naming the class that stopped it, and steps ending at gate with no respond.

POST /api/v1/assistant/threads/{thread_id}/release – HTTP 200, inspector only. Where the gate held a run, this sends the real answer; the released run carries held cleared and respond on the end of its steps.

POST /api/v1/assistant/threads/nobody-started-this/release – HTTP 404.

```
{"detail":"there is no run on thread nobody-started-this to release"}
```

The wording of every model-written line differs between runs. The figures the reply quotes, the tools the plan chose, the class the champion read and the steps the run recorded do not.

THE SERVICE'S OWN RECORD

Uvicorn reports each call as it is served. The request log of Week 3 Day 2 writes a line of its own into output/logs/requests.log — clock reading, method, path, status, milliseconds, pipe-separated — and the audit trail of Week 3 Day 3 writes one row per case move into output/audit_trail.csv.

```
1786648785.787|GET|/api/v1/health|200|2.1
1786648786.119|POST|/api/v1/accounts|201|321.3
1786648786.357|POST|/api/v1/accounts/login|200|226.7
1786648786.457|POST|/api/v1/readings|201|32.4
1786648786.475|POST|/api/v1/readings|201|6.2
1786648786.494|GET|/api/v1/readings/RDG50000002|200|7.4
1786648786.512|GET|/api/v1/readings|200|6.2
1786648786.525|GET|/api/v1/readings/RDG99999999|404|2.7
1786648786.542|PATCH|/api/v1/readings/RDG50000002/status|200|8.4
1786648786.555|PATCH|/api/v1/readings/RDG50000002/status|409|3.5
1786648786.568|POST|/api/v1/screening|401|4.0
1786648797.834|POST|/api/v1/screening|200|784.7
1786648797.935|POST|/api/v1/assistant/questions|503|90.8
1786648797.948|POST|/api/v1/assistant/threads/nobody-started-this/release|404|2.3
recorded_at,actor,role,reading_id,from_status,to_status
2026-08-13T19:19:46.541715+00:00,je/1180,inspector,RDG50000002,raised,assigned
```

Every call is logged, whether answered or refused — the 401, 404, 409 and 503 are all there beside the successes.

IMPLEMENTATION STEPS

Step 1 – Move into the project directory. Every command in this document is run from here, and every path in it is relative to it.

```
cd Project
```

Step 2 – Install the dependencies. The libraries that do the arithmetic are pinned to exact versions, so the figures printed in this document reproduce on your machine.

```
pip install -r requirements.txt
```

Step 3 – Put the language model settings into .env at the project root. The file is supplied with both values empty, and both have to be filled in before any module that reaches the model will run. A free-tier key is obtained from the Google AI Studio console.

```
GEMINI_API_KEY=your_api_key_here
GEMINI_MODEL=gemini-3.1-flash-flash
```

Step 4 – Run each day's module as it is written. A script is run by its path and a package module with the -m switch, always from the project root.

```
python3 scripts/day01_bill_from_reading.py
python3 -m metermind.core.entities
```

Step 5 – Run the days in order the first time through. A day reads what earlier days wrote, so Week 4 Day 5 cannot sort readings into queues before Week 4 Day 4 has saved a champion, and Week 7 Day 2 cannot answer before Week 7 Day 1 has built the store. The OUTPUT section above states which day produces which file. Week 7 Day 1 downloads the embedding model on its first run, which is roughly ninety megabytes.

Step 6 – Serve the platform.

```
python3 -m metermind.api.application_factory
```

Step 7 – Try the service. The interactive documentation is at http://127.0.0.1:8000/docs, where the Authorize button accepts the token the login returns. Everything added from Week 3 Day 4 on requires it — the listing, the case move, the screening and the assistant.

```
curl -s -X POST http://127.0.0.1:8000/api/v1/accounts \
  -H "Content-Type: application/json" \
  -d '{"account_number":"ACC10000123","email_address":"meena@example.in",
       "password":"Chennai#2026","role":"inspector"}'

TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/accounts/login \
  -H "Content-Type: application/json" \
  -d '{"account_number":"ACC10000123","password":"Chennai#2026"}' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['access_token'])")

curl -s -X POST http://127.0.0.1:8000/api/v1/assistant/questions \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"question":"Why is reading RDG20000011 billed at that amount?",
       "consumer_category":"domestic","billing_period":"2025-05"}'
```

Step 8 – Run the testcases. Every day adds between three and five of them, and the whole file is present from the first day. Finish a day and that day's testcases and every earlier day's pass together, while the rest fail because those days have not been written yet. The whole file runs in under a minute, and reaches one hundred and seventy-eight passing on Week 8 Day 3.

```
python3 -m pytest tests.py -v
```

FINAL OUTPUT

Original given as two screenshot images of the terminal (final pytest run). Per the spec's own text under Step 8: the whole test file runs in under a minute and reaches 178 tests passing once Week 8 Day 3 is complete — that is the terminal state these two images show.